Université de Montpellier

Master Informatique

Architectures Logicielles pour l'IA Agentique

Auteurs

Sofiane Hammar
Anouar Soufyani
Souad Hadji

Encadrant

M. Abdelhak-Djamel Seriai

Année universitaire 2025–2026

Résumé

Ce rapport retrace les travaux que nous avons menés dans le cadre du TER de Master 1 Informatique. Le sujet porte sur les architectures agentiques : comment combiner les Systèmes Multi-Agents classiques et les grands modèles de langage pour construire un système réellement utile.

La première partie est un état de l'art sur les SMA et l'Agentic AI. On y compare dix-sept études pour comprendre ce que chaque approche apporte et où elle est limitée. Cette analyse nous a guidés pour choisir LangGraph comme outil d'orchestration.

La deuxième partie décrit le prototype que nous avons développé : un **système de recrutement automatisé multi-agents**. Il comprend dix agents spécialisés, une mémoire vectorielle persistante (ChromaDB), un point de contrôle humain obligatoire, deux modes de déploiement (monolithique et microservices Docker), une API REST avec flux en temps réel, une interface web NiceGUI et 97 tests automatisés.

Table des matières

Résumé **1**

1 Introduction **5**
1.1 Contexte 5
1.2 Problématique 5
1.3 Objectifs du TER 6
1.4 Organisation du rapport 6
1.5 Répartition du travail dans le temps 7

2 État de l'art **8**
2.1 Les Systèmes Multi-Agents 8
2.2 L'Agentic AI 9
2.3 Différences et complémentarité 11
2.4 Ce que nous avons retenu pour notre projet 12

3 Genèse et conception du projet **13**
3.1 Objectif pédagogique 13
3.2 Sujets envisagés 13
3.3 Un projet Agentic AI avec compétences SMA 14
3.4 Cahier des charges 14

4 Choix du framework : LangGraph **15**
4.1 Méthode de sélection 15
4.2 Premier filtre : comparaison binaire des neuf frameworks 16
4.3 Comparaison approfondie des quatre candidats 16
4.4 Pourquoi LangGraph 18

5 Implémentation **19**
5.1 Organisation du projet 19
5.2 Architecture générale 20
5.3 Le blackboard : GraphState 20

5.4 Construction du graphe 21
5.5 Rôle du LLM dans le système 24
5.5.1 Principe : LLM pour le cognitif, déterministe pour le structurel 24
5.5.2 Cartographie des appels LLM par agent 25
5.5.3 Choix du fournisseur : pourquoi Groq et pas Ollama ou un autre 27
5.5.4 Implémentation centralisée : `get_llm()` et gestion du quota . . 27
5.6 Les agents et leurs choix de conception 29
5.6.1 A1 : Orchestrateur 29
5.6.2 A2 : Analyste de poste 29
5.6.3 A3 : Pipeline de collecte (trois sous-agents) 30
5.6.4 A6 : Déduplicateur 31
5.6.5 A4 : Évaluateur ($\times N$ parallèles) 32
5.6.6 A5 : Vérificateur ($\times N$ parallèles) 32
5.6.7 Nœud injection RAG 33
5.6.8 A7 : Recruteur 33
5.6.9 A8 : Persistance RAG 35
5.7 Mémoire RAG 36
5.8 Pipeline de sourcing multi-sources (A3) 36
5.8.1 A3a : Stratège : génération du plan de recherche 36
5.8.2 A3b : Collecteur : exécution multi-sources 37
5.8.3 A3c : Filtre anti-bruit : quatre étapes algorithmiques 38
5.9 Feature Model 38
5.10 API Gateway 38
5.11 Interface graphique NiceGUI 40
5.12 Deux architectures d'exécution 43
5.12.1 Mode principal : LangGraph monolithique 43
5.12.2 Mode secondaire : microservices Docker 44
5.12.3 Comparaison architecturale 47
5.13 Tests automatisés 47

6 Évaluation et résultats 52
6.1 Run de référence (Run 1) — base ChromaDB vide 52
6.2 Run 2 — premier apport du RAG (base peuplée) 53
6.3 Run 3 — accumulation de la mémoire 55
6.4 Synthèse comparative des trois runs 58

7 Conclusion et perspectives 59
7.1 Bilan du projet 59
7.2 Ce qu'on a appris 62
7.3 Limites 62

7.4 Considérations éthiques et RGPD 63

Liste des tableaux

2.1 SMA classiques vs Agentic AI 11

4.1 Comparatif binaire des neuf frameworks (✓ = supporté, × = absent) . 16
4.2 Scores par critère (note /5, total /55) 17

5.1 Agents LLM vs agents déterministes dans le pipeline 25
5.2 Collections ChromaDB 36
5.3 Feature model, variabilité architecturale 38
5.4 Endpoints de l'API Gateway 39
5.5 Services Docker (`docker-compose.microservices.yml`) 45
5.6 LangGraph monolithique vs microservices Docker 47
5.7 97 tests répartis sur 7 fichiers 48

6.1 Métriques du Run 1 — base ChromaDB vide 52
6.2 Meilleurs candidats identifiés au Run 1 53
6.3 Métriques du Run 2 — base ChromaDB peuplée (Run ID : 20260523_022804) 53
6.4 Candidats identifiés au Run 2 (valides et douteux) 54
6.5 Métriques du Run 3 — base ChromaDB enrichie (Run ID : 20260523_024521) 56
6.6 Évolution des métriques sur trois runs successifs — même fiche de poste 58

7.1 Correspondance entre concepts SMA/Agentic AI et implémentation . . 61

Chapitre 1

Introduction

1.1 Contexte

Depuis 2022, les grands modèles de langage (LLM) ont changé la façon de concevoir des logiciels intelligents. On ne les utilise plus seulement pour générer du texte : on les intègre dans des systèmes capables de planifier, d'appeler des outils et de prendre des décisions. Ces systèmes sont appelés **systèmes agentiques**.

Ce TER se place à deux niveaux : utiliser l'IA pour automatiser un processus métier (le recrutement), et appliquer les bonnes pratiques du génie logiciel pour que ce système soit fiable, testable et maintenable.

1.2 Problématique

Dans la plupart des entreprises, chercher des candidats et faire une première sélection reste un travail manuel et répétitif. Un recruteur passe beaucoup de temps à parcourir des profils sur plusieurs plateformes, supprimer les doublons, filtrer selon des critères simples et rédiger des premiers messages de contact. Ce sont des tâches qu'on peut automatiser, mais pas n'importe comment : si on délègue tout à une machine, on perd la traçabilité des décisions et on s'expose à des problèmes de confidentialité.

La question centrale de ce TER est donc :

> *Comment automatiser les tâches répétitives du recrutement tout en gardant un humain dans la boucle pour les décisions importantes, et en garantissant que chaque étape est traçable et que les données personnelles sont protégées ?*

Cette question oriente directement l'architecture. Il faut un point d'arrêt humain ex-

plicite avant tout contact avec un candidat. Les agents doivent être spécialisés pour que chaque décision soit attribuable à un composant précis. Les transmissions externes sont limitées à ce qui est nécessaire : Groq pour l'inférence LLM, DuckDuckGo pour les recherches web, et les APIs GitHub et Stack Overflow pour les sources techniques.

1.3 Objectifs du TER

Ce TER a trois objectifs.

Objectif théorique. Comprendre ce que sont les Systèmes Multi-Agents et l'Agentic AI, voir ce qu'ils ont en commun et ce qui les distingue. On a pour cela analysé dix-sept études rassemblées dans le répertoire `Etudes/`.

Objectif méthodologique. Choisir un framework d'orchestration adapté au projet. On a comparé neuf solutions (LangGraph, AutoGen, CrewAI, LangChain, LlamaIndex, MetaGPT, OpenAI Swarm, AWS Strands, Semantic Kernel) selon des critères précis : graphe orienté, état partagé typé, HITL natif, intégration RAG, indépendance vis-à-vis des fournisseurs LLM, et maturité du projet.

Objectif applicatif. Concevoir et développer un vrai système de recrutement multi-agents. Le résultat est un prototype avec dix agents spécialisés (A1 à A8, A3 étant lui-même divisé en A3a, A3b, A3c), une mémoire vectorielle inter-runs (ChromaDB), un point de contrôle humain obligatoire, une API REST, une interface web et 97 tests automatisés.

1.4 Organisation du rapport

Le chapitre 2 pose les bases théoriques sur les SMA et l'Agentic AI. Le chapitre 4 explique pourquoi on a choisi LangGraph. Le chapitre 3 retrace comment le sujet a été défini. Le chapitre 5 décrit l'implémentation en détail. Le chapitre 6 présente les résultats obtenus. Enfin, le chapitre 7 conclut et ouvre des pistes d'amélioration.

1.5 Répartition du travail dans le temps

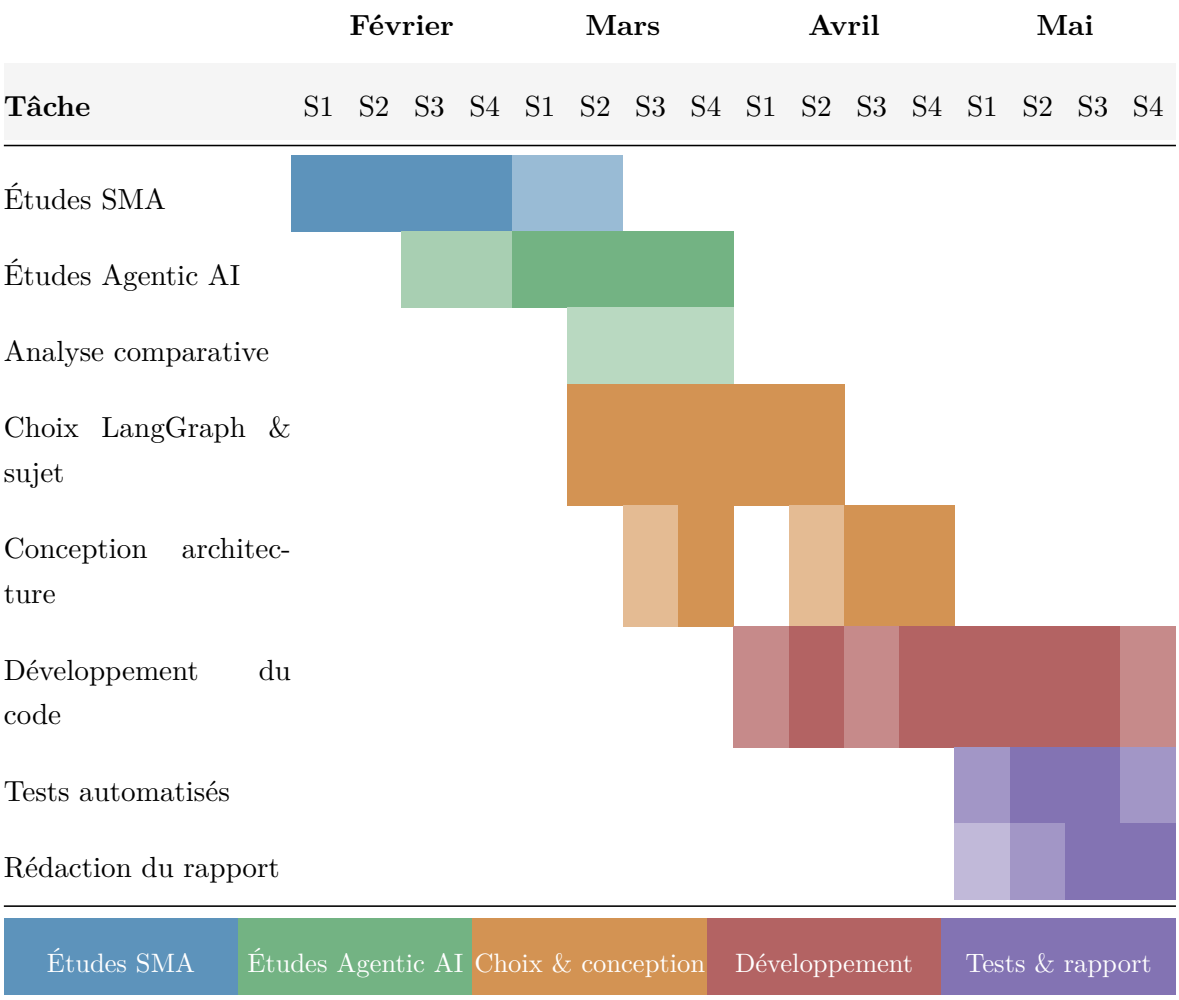


FIGURE 1.1 – Répartition du travail sur seize semaines (février–mai 2026).

Chapitre 2

État de l'art

Notre encadrant M. Seriai nous a fourni dix-sept études : six sur les Systèmes Multi-Agents classiques et onze sur l'Agentic AI. Nous avons rédigé un bilan individuel pour chacune (consultable dans le dossier `Etudes/`), puis conduit notre propre analyse comparative entre les deux paradigmes. Ce chapitre présente les conclusions de ce travail et ce que nous en avons retenu pour notre architecture.

2.1 Les Systèmes Multi-Agents

Un Système Multi-Agents est un ensemble d'agents autonomes qui interagissent dans un environnement partagé. Ce qui le distingue d'un programme distribué classique, c'est que chaque agent prend ses décisions de façon indépendante : il n'y a pas de chef d'orchestre central. Si un agent tombe en panne, le reste du système continue de fonctionner [6].

Chaque agent a quatre propriétés de base. Il est **autonome** (il agit sans attendre d'instructions à chaque étape), **réactif** (il perçoit son environnement et y répond), **proactif** (il prend des initiatives) et **sociable** (il communique avec les autres agents). On décrit généralement un SMA selon quatre dimensions : les agents, l'environnement, leurs interactions et les règles qui organisent leurs rôles [2].

Architectures et types d'agents. Les SMA peuvent s'organiser selon trois structures [2] : l'architecture **centralisée** (un agent maître coordonne les autres), **décentralisée** (la coordination émerge des interactions locales) ou **hiérarchique** (supervision globale et autonomie locale combinées). Notre prototype suit un modèle hiérarchique, avec un agent superviseur (A1) qui coordonne le pipeline et produit le rapport final.

Les agents se distinguent également par leur niveau cognitif : **réactifs** (réponse directe

aux stimuli, sans état interne), **cognitifs** (planification et raisonnement interne) ou **hybrides** (combinaison des deux). Les agents LLM de notre système relèvent de la catégorie cognitive.

Coordination, coopération et négociation. Trois mécanismes fondamentaux structurent les échanges entre agents [1] : la **coopération** (agents poursuivant un but commun), la **coordination** (organisation des actions pour éviter les conflits) et la **négociation** (résolution distribuée des désaccords). Ces mécanismes s'appuient sur des protocoles formels éprouvés : le **Contract Net Protocol** (appel d'offres entre agents), la **théorie des jeux** (modélisation des équilibres) et **FIPA-ACL** (Foundation for Intelligent Physical Agents – Agent Communication Language, standard de communication inter-agents) [3]. La force de ces approches réside dans leurs fondements formels : les règles sont explicites, les transitions déterministes, les décisions auditables.

Le modèle BDI. L'architecture cognitive de référence dans les SMA est le modèle **BDI** (Beliefs, Desires, Intentions). Un agent BDI maintient une représentation de ce qu'il sait sur le monde (*beliefs*), ce qu'il veut atteindre (*desires*), et le plan concret qu'il est en train d'exécuter (*intentions*) [1]. Ce modèle nous a directement inspiré : dans notre système, les *beliefs* correspondent au `GraphState` partagé, les *desires* à la mission de chaque agent, et les *intentions* aux appels LLM et aux écritures dans l'état partagé.

Émergence et limites. Une propriété remarquable des SMA est l'**émergence** : des comportements collectifs complexes naissent d'interactions locales simples, sans qu'aucun agent n'ait planifié le résultat global [5]. Les SMA modernes intègrent aussi du Multi-Agent Reinforcement Learning pour permettre aux agents d'adapter leurs stratégies par apprentissage. Cette approche introduit des défis propres au MARL : non-stationnarité de l'environnement, instabilité de convergence et tension entre comportements coopératifs et compétitifs [4]. Ces défis ont directement guidé certains choix de conception de notre prototype.

La limite principale des SMA classiques est leur rigidité face au langage naturel : ils fonctionnent bien sur des données structurées, mais peinent dès qu'il faut interpréter une fiche de poste rédigée librement. C'est là qu'intervient l'Agentic AI.

2.2 L'Agentic AI

L'Agentic AI est différente d'un simple chatbot. Un LLM conversationnel répond à une question et s'arrête là. Un système agentique, lui, enchaîne une série d'actions, appelle des outils, observe les résultats et adapte sa stratégie en cours de route [7]. Ce n'est

pas juste une amélioration : c'est un changement de catégorie, du texte généré vers un système qui *agit*.

Pour comprendre où se situe l'Agentic AI, il faut la replacer dans une trajectoire d'évolution [10] :

GenAI (génère du texte ou du code sur demande, sans mémoire ni suite) → **AI Agents** (exécute des tâches précises avec des outils) → **Agentic AI** (planifie, mémorise, s'adapte sur plusieurs étapes) → **Communautés agentiques** (plusieurs agents IA et humains coordonnés par des règles formelles)

Notre système se situe entre l'Agentic AI et les communautés agentiques : plusieurs agents spécialisés coordonnés, avec un humain intégré dans la boucle pour les décisions critiques.

On distingue généralement trois niveaux dans ce paradigme. Les **AI Agents** exécutent des tâches précises en utilisant le *prompt engineering* et en accédant à des outils comme des APIs ou des bases de données. L'**Agentic AI** proprement dite ajoute la planification autonome multi-étapes, la mémoire persistante et l'adaptation dynamique aux résultats. Les **communautés agentiques** constituent le niveau le plus avancé : plusieurs agents IA et humains coexistent dans une structure gouvernée par des règles formelles [12].

Comment un agent est construit. Un système agentique repose sur quatre composantes : un **LLM** comme moteur cognitif, des **outils** (APIs, bases de données, scraping) pour agir sur le monde réel, une **mémoire** à deux niveaux (contexte court terme dans la fenêtre du LLM, mémoire long terme via RAG vectoriel), et un **prompt système** qui définit le rôle et les contraintes de l'agent [10]. À chaque action, l'agent suit une boucle en quatre phases : (1) **Analyse** de la demande, (2) **Planification** de la stratégie et découpage en sous-tâches, (3) **Sélection des outils** nécessaires, (4) **Exécution** et ajustement selon les résultats obtenus. On peut situer tout système sur une échelle de maturité agentique de 0 à 4 [10] : niveau 0 (chatbot passif, réponse unique), niveau 1 (chaîne d'appels LLM sans mémoire), niveau 2 (agent avec mémoire et outils), niveau 3 (orchestration autonome multi-agents avec gouvernance humaine), niveau 4 (agent totalement autonome, capable de se reconfigurer). Notre système se situe au niveau 3 : orchestration autonome de plusieurs agents spécialisés avec point d'arrêt humain intégré.

Le double paradigme. Un éclairage important que nous avons retenu de nos lectures est la distinction entre deux approches fondamentalement différentes [11]. L'approche **symbolique** repose sur la planification déterministe, l'état persistant et la véri-

fication formelle, elle domine dans des domaines où la prédictibilité est non négociable comme la santé critique. L'approche **neuronale** s'appuie sur les LLM, la génération probabiliste et une forte adaptabilité, elle s'impose dans les environnements dynamiques. La tendance convergente identifiée dans les études est une architecture **neuro-symbolique hybride** : LLM pour la flexibilité cognitive, composants déterministes pour les décisions critiques. Cette trajectoire a directement orienté notre conception.

Les enjeux transversaux. Plusieurs défis structurels apparaissent dans l'ensemble des études analysées. Le premier est l'**hallucination** : les LLM peuvent produire des informations incorrectes avec une apparente confiance, et deux appels identiques peuvent donner des résultats différents [8]. Le second est la **sécurité** : les agents sont exposés aux attaques par *prompt injection* (manipulation des instructions via des données externes) et aux fuites de données confidentielles [9]. Le troisième est l'**explicabilité** : contrairement aux SMA classiques dont les règles sont traçables, les décisions d'un LLM sont difficiles à justifier formellement. Ces trois enjeux ont directement influencé notre conception : les guards déterministes d'A5, le HITL obligatoire et la journalisation par nœud sont des réponses directes à ces risques.

2.3 Différences et complémentarité

Les deux paradigmes partagent les mêmes fondements : autonomie des agents, poursuite d'objectifs, coordination multi-entités, mémoire persistante. Ce sont leurs *mécanismes* qui divergent radicalement.

Table 2.1 – SMA classiques vs Agentic AI

Dimension	**SMA classiques**	**Agentic AI**
Raisonnement	Logique, déterministe, prouvable	Probabiliste (LLM), flexible
Coordination	Protocoles formels (ACL, Contract Net)	Orchestration par prompts
Type d'entrées	Données structurées	Langage naturel hétérogène
Explicabilité	Inhérente, règles traçables	Déficit structurel
Maturité	Décennies, fondements formels	Écosystème encore fragmenté
Point fort	Robustesse, auditabilité	Compréhension sémantique
Point faible	Rigide face au langage naturel	Hallucinations, non déterministe

Les deux approches ne sont pas opposées : elles se complètent. Les SMA apportent ce qui manque à l'Agentic AI : une coordination formelle, un état partagé bien structuré et des mécanismes de validation vérifiables. Les LLM apportent à leur tour ce que les SMA classiques ne savent pas faire : comprendre et produire du langage naturel.

Un pipeline agentique sans structure formelle produit facilement des résultats imprévisibles. En imposant un état typé, une validation par un second agent indépendant et un routage basé sur des seuils numériques, on obtient un système qui reste flexible mais aussi auditable. C'est l'approche formalisée dans le catalogue de 46 patterns architecturaux proposé par Milosevic et Rabhi [12].

2.4 Ce que nous avons retenu pour notre projet

Notre système de recrutement applique concrètement cette complémentarité. On utilise les LLM là où ils sont utiles : comprendre une fiche de poste (A2), générer des requêtes de recherche (A3a), noter un candidat (A4), vérifier les évaluations (A5), rédiger des messages de contact (A7). Pour tout le reste, on applique des structures SMA :

— **Blackboard** : le `GraphState` typé est l'espace de communication partagé entre tous les agents, avec des droits d'écriture séparés.
— **Superviseur** : A1 coordonne le flux sans exécuter de tâche métier.
— **Validation pair-à-pair** : A4 évalue, A5 contrôle indépendamment — chaque agent ne lit que ses propres champs.
— **Routage déterministe** : la décision de contacter un candidat repose sur des seuils numériques explicites, pas sur un choix du LLM.
— **Human-in-the-Loop** : aucune action irréversible sans validation humaine.

C'est cette combinaison, la flexibilité des LLM encadrée par la rigueur des SMA, qui forme la base théorique de notre système. Avant de le construire, on a dû choisir le bon outil d'orchestration : c'est ce que le chapitre suivant explique.

Chapitre 3

Genèse et conception du projet

3.1 Objectif pédagogique

Ce TER avait une ambition précise : ne pas se contenter d'un système agentique générique, mais démontrer qu'une architecture Agentic AI peut intégrer des **compétences formelles issues des SMA classiques**. Le projet devait être à la fois un système Agentic AI opérationnel et une instanciation concrète des patterns SMA étudiés.

3.2 Sujets envisagés

Plusieurs domaines ont été explorés avant le choix final :

Comparateur de prix par scraping. Agents scraper par site, normalisateur, comparateur, alerteur. Écarté : valeur ajoutée du LLM marginale, pipeline trop linéaire, pas de besoin naturel de HITL ni de mémoire inter-runs.

Assistant de veille technologique. Agents par domaine agrégeant flux RSS, arXiv, GitHub Trending. Écarté : périmètre difficile à délimiter, évaluation qualitative subjective, métriques non objectives.

Système de recommandation personnalisée. Agents modélisant différents aspects du profil utilisateur, convergence par vote. Écarté : nécessite des données historiques difficiles à simuler dans un prototype.

Système de recrutement automatisé multi-agents. Lors d'un rendez-vous avec notre encadrant, nous avons réfléchi et discuté des sujets envisagés. Cette piste a émergé de cet échange : un pipeline d'agents pour sourcer, évaluer et contacter des candidats.

Elle a été retenue car elle répond à tous les critères que nous cherchions : pipeline naturellement multi-étapes, HITL obligatoire avant tout contact, RAG utile dès le deuxième run, métriques objectives (scores, taux de bruit, durée). C'est ce sujet qui a été choisi lors de ce rendez-vous.

3.3 Un projet Agentic AI avec compétences SMA

Le sujet retenu est conçu comme une démonstration de convergence : le **pattern Superviseur** (A1 coordonne), le **Blackboard** (`GraphState` typé), le **Map-Reduce** (A4/A5 via `Send()`), la **validation pair-à-pair** (A4 évalue, A5 contrôle indépendamment), et le **HITL** comme gouvernance humaine.

Le recrutement automatisé est le scénario applicatif ; l'Agentic AI avec compétences SMA en est l'architecture.

3.4 Cahier des charges

EF-1 Sourcer des profils sur des sources publiques sans clé API payante.

EF-2 Filtrer le bruit sans perdre de profils valides.

EF-3 Évaluer chaque profil avec score et explications.

EF-4 Contrôle pair-à-pair : A5 peut invalider ou ajuster le score de A4.

EF-5 Bloquer tout contact sans validation humaine explicite (HITL).

EF-6 Mémoire inter-runs calibrée par fiche de poste (RAG).

EF-7 Suivi en temps réel via streaming SSE.

Chapitre 4

Choix du framework : LangGraph

4.1 Méthode de sélection

Pour structurer notre comparaison, nous nous sommes appuyés sur l'étude de Vaidhyanathan et Taibi [13], qui proposent une grille d'évaluation multidimensionnelle articulée autour de cinq axes (gestion des objectifs, planification, patterns architecturaux, gestion d'état et mémoire, maturité). Nous avons adapté cette grille à notre projet en ne retenant que les critères directement imposés par nos besoins, et en complétant par notre propre comparatif détaillé (`Etudes/comparatif_frameworks-2.numbers`).

Les **cinq critères non négociables** retenus, chacun justifié par une exigence concrète du projet :

Architecture en graphe orienté
: Notre pipeline nécessite des branches conditionnelles (routage par score), du parallélisme (évaluation simultanée de plusieurs candidats) et des boucles. Une architecture séquentielle est insuffisante.

État partagé typé
: Dix agents doivent se transmettre des données structurées (fiche de poste, profils, scores, validations). Un état typé garantit que chaque agent reçoit exactement ce qu'il attend.

HITL natif
: Aucun message de contact ne doit partir sans validation humaine. Le mécanisme d'interruption doit être intégré au framework, pas contourné.

Intégration RAG
: La mémoire vectorielle (ChromaDB) est centrale dans notre architecture. Le framework doit s'y connecter sans couche supplémentaire.

Indépendance fournisseur LLM

Nous utilisons Groq en développement mais devons pouvoir basculer vers un modèle local (Ollama) sans toucher au code des agents.

4.2 Premier filtre : comparaison binaire des neuf frameworks

Nous avons évalué neuf frameworks candidats sur ces cinq critères. Le tableau suivant montre lesquels les satisfont :

TABLE 4.1 – Comparatif binaire des neuf frameworks (✓ = supporté, × = absent)

Framework	Graphe	État typé	HITL	RAG	Indép.	Maturité	Verdict
LangGraph	✓	✓	✓	✓	✓	élevée	**Candidat**
AutoGen	×	×	✓	partiel	✓	moyenne	Candidat partiel
CrewAI	×	limité	×	partiel	✓	moyenne	Candidat partiel
CAMEL	×	×	×	partiel	✓	faible	Candidat partiel
LangChain	×	limité	partiel	✓	✓	élevée	Écarté
LlamaIndex	×	×	×	✓	✓	élevée	Écarté (RAG seul)
MetaGPT	SOP	×	×	×	✓	moyenne	Écarté
AWS Strands	partiel	partiel	partiel	✓	×	moyenne	Écarté
Semantic Kernel	×	✓	✓	✓	×	élevée	Écarté
OpenAI Swarm	×	×	×	×	×	faible	Écarté

Cinq frameworks sont écartés immédiatement : **LangChain** n'a ni graphe ni checkpointer — sa structure est linéaire, insuffisante pour des topologies complexes. Retenu uniquement comme couche `init_chat_model`. **LlamaIndex** est un moteur RAG, pas un orchestrateur multi-agents — ses agents ne communiquent pas entre eux et ne prennent pas de décisions indépendantes. **MetaGPT** suit des pipelines PM→Dev→QA figés, inadaptés au recrutement. **AWS Strands** impose un lock-in AWS incompatible avec la reproductibilité locale. **Semantic Kernel** impose Azure. **OpenAI Swarm** est un outil pédagogique explicitement déconseillé en production par OpenAI.

Restent quatre candidats sérieux : LangGraph, AutoGen, CrewAI et CAMEL.

4.3 Comparaison approfondie des quatre candidats

Nous avons évalué ces quatre frameworks sur onze critères notés de 1 à 5. Les scores reposent sur notre propre comparatif détaillé (`Etudes/comparatif_frameworks-2.numbers`), lui-même construit à partir de recherches documentaires sur chaque framework (docu-

mentation officielle [16, 14, 15], articles techniques, retours d'expérience) et de la grille méthodologique de Vaidhyanathan et Taibi [13] :

TABLE 4.2 – Scores par critère (note /5, total /55)

Critère	**LangGraph**	**AutoGen**	**CrewAI**	**CAMEL**
Architecture de base	**5**	4	4	3
Coordination entre agents	**5**	4	4	3
Support multi-agents	**5**	5	5	4
Intégration principes SMA	**5**	4	4	4
Communication inter-agents	**5**	5	4	4
Négociation & adaptation	4	**5**	3	3
Mémoire & état	**5**	4	4	3
Extensibilité	**5**	5	4	4
Facilité d'apprentissage	2	3	**5**	3
Maturité production	**5**	4	**5**	2
Support LLM multiple	**5**	**5**	**5**	4
Total /55	**51**	48	47	37

LangGraph (51/55) obtient le meilleur score global. Son seul point faible est la courbe d'apprentissage (2/5), coût assumé pour un projet de plusieurs semaines où la puissance expressive prime.

AutoGen (48/55) est fort sur la négociation et le support LLM, mais son modèle conversationnel rend le routage par score numérique difficile à auditer. Le fan-out parallèle n'est pas natif.

CrewAI (47/55) est le plus facile à prendre en main et excellent en production, mais sa coordination est LLM-first : la séquence est décidée par le modèle, pas par le code. Cela rend le comportement moins prévisible pour un pipeline de recrutement où chaque décision doit être traçable.

CAMEL (37/55) est conçu pour la recherche académique sur les comportements multi-agents. Sa maturité en production est faible et il n'est pas adapté à un pipeline industriel.

4.4 Pourquoi LangGraph

LangGraph est le seul framework qui répond à l'ensemble de nos contraintes sans qu'on ait à le contourner ou à compléter ses lacunes avec du code externe.

Le premier argument est la traçabilité du flux. Dans notre pipeline, la décision de contacter un candidat repose sur un score numérique, pas sur un choix du modèle de langage. LangGraph permet de coder cette logique directement dans le graphe via `add_conditional_edges` : le comportement est prévisible, auditable, et identique à chaque exécution. AutoGen, dont la coordination passe par des messages texte, ne donne pas cette garantie.

Le deuxième argument est le parallélisme natif. Évaluer dix candidats séquentiellement prend 30 à 40 secondes. Avec `Send()`, LangGraph lance les dix évaluations simultanément et fusionne automatiquement les résultats via les reducers du graphe. Ce mécanisme Map-Reduce est natif et ne nécessite aucune infrastructure supplémentaire. CrewAI propose du parallélisme mais sans la gestion d'état typée qui va avec.

Le troisième argument est le contrôle humain. Notre cahier des charges impose qu'aucun message de contact ne parte sans validation humaine. `interrupt_before` suspend le pipeline à un nœud précis, conserve l'état complet en mémoire, et permet de reprendre exactement là où le pipeline s'était arrêté après la décision du recruteur. Aucun autre framework parmi ceux évalués ne propose ce mécanisme nativement : on aurait dû le recoder manuellement, avec tous les risques que cela implique.

Enfin, le quatrième argument est la persistance. `MemorySaver` sauvegarde un snapshot de l'état après chaque nœud. En cas de timeout Groq ou de panne réseau, le pipeline reprend depuis le dernier nœud terminé sans tout recommencer. C'est essentiel pour un processus qui peut durer plusieurs minutes.

Ainsi, LangGraph s'est imposé comme le framework le plus pertinent pour notre projet. Il combine orchestration déterministe, exécution parallèle, persistance d'état et validation humaine native, tout en limitant la complexité d'implémentation. Aucun des neuf frameworks évalués ne réunit ces quatre propriétés sans nécessiter de développements additionnels.

Chapitre 5

Implémentation

5.1 Organisation du projet

Avant de décrire les composants, voici l'arborescence complète du projet qui en reflète l'architecture :

Listing 5.1 – Arborescence du projet TER_M1_Agentic_AI/

```
src/
  agents/
    orchestrateur.py        # A1 : superviseur + rapport final
    analyste.py             # A2 : fiche de poste -> JSON structure
    chercheur_stratege.py   # A3a : plan de requetes par source
    chercheur_collecteur.py# A3b : collecte DDG / GitHub / SO
    chercheur_filtre.py     # A3c : filtre anti-bruit algorithmique
    deduplicateur.py        # A6 : fusion profils
    evaluateur.py           # A4 : scoring LLM (N instances
       paralleles)
    verificateur.py         # A5 : verification + guards
       deterministes
    recruteur.py            # A7 : redaction messages de contact
    persistance.py          # A8 : persistance ChromaDB
    injection_rag.py        # noeud injection proactive inter-runs
  tools/
    rag.py                  # ChromaDB : blacklist, cache, injection
    scraping.py             # scraping HTTP + BeautifulSoup 4
    search.py               # DDG, GitHub API, Stack Exchange API
    github_api.py           # client GitHub Search Users API
    stackoverflow_api.py    # client Stack Exchange API
  ui/
    app.py                  # NiceGUI : 6 pages
    api_client.py           # client HTTP vers l'API REST
    layout.py               # composants visuels partages
```

```
  graph.py                    # assemblage du graphe LangGraph
  state.py                    # GraphState TypedDict + reducers
  config.py                   # constantes, get_llm()
  prompts.py                  # prompts systeme de chaque agent
  api.py                      # FastAPI : 10 endpoints + SSE
  main.py                     # point d'entree CLI
  observabilite.py            # collecte metriques JSON par noeud
  logger.py                   # configuration logging unifie
services/
  orchestrateur/main.py       # orchestrateur microservices HTTP
tests/
  test_graph.py               # 17 tests topologie et routage
  test_api.py                 # 22 tests endpoints REST
  test_filtre.py              # 24 tests filtres URL/keywords
  test_rag.py                 # 16 tests ChromaDB
  test_deduplicateur.py       # 11 tests fusion profils
  test_verificateur_rules.py  # 5 tests guards A5
  test_stratege_queries.py    # 2 tests requetes A3a
data/chromadb/                # base vectorielle locale (persistante)
docker-compose.microservices.yml
run_local.sh                  # lanceur tout-en-un mode local
requirements.txt
.env.example
```

Le projet suit une séparation stricte en trois couches : la couche **orchestration** (`graph.py`, `state.py`, `agents/`), la couche **service** (`api.py`, FastAPI), et la couche **présentation** (`ui/`, NiceGUI). Cette séparation garantit que l'interface ne connaît pas LangGraph et que les agents ne connaissent pas l'API.

5.2 Architecture générale

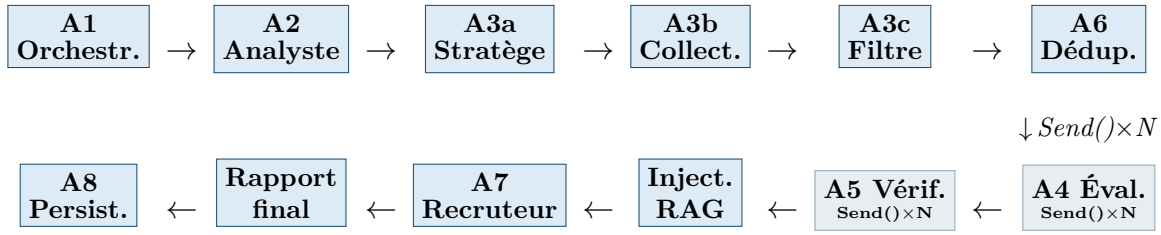


FIGURE 5.1 – Pipeline complet du SMA de recrutement (16 nœuds LangGraph).

5.3 Le blackboard : GraphState

Listing 5.2 – src/state.py : blackboard typé avec reducers

```python
class GraphState(TypedDict):
    fiche_poste: str                                    # Input utilisateur
    profil_competences: dict                            # A2 ecrit
    requetes_recherche: dict                            # A3a ecrit
    resultats_bruts: list[dict]                         # A3b ecrit
    profils_bruts: Annotated[list[Candidat], operator.add]      # A3c
    profils_dedupliques: list[Candidat]                 # A6 ecrit
    candidats_scores: Annotated[list[CandidatScore], operator.add]  # A4xN
    candidats_valides: Annotated[list[CandidatValide], operator.add] # A5xN
    messages_envoyes: Annotated[list[dict], operator.add]       # A7
    rapport_final: str                                  # A1 ecrit
    messages: Annotated[list, add_messages]             # trace LangGraph interne
```

Les champs avec `Annotated[list, operator.add]` sont des **reducers**. Quand plusieurs instances d'A4 tournent en parallèle et écrivent chacune un résultat, LangGraph les fusionne automatiquement dans la liste sans conflit.

On a choisi un `StateGraph` typé plutôt qu'un `MessageGraph` parce que notre pipeline ne transmet pas de messages conversationnels mais des artefacts métier (profils, scores, validations). Le typage fort permet aussi de détecter les erreurs à la compilation plutôt qu'à l'exécution.

5.4 Construction du graphe

Le graphe est construit en trois étapes : déclaration des nœuds, câblage des arêtes, puis compilation avec le checkpointer et le point d'arrêt HITL.

Étape 1 — déclaration des nœuds. Chaque nœud est une fonction Python ordinaire. `add_node("nom", fonction)` enregistre la fonction dans le graphe et lui attribue un identifiant. LangGraph appellera cette fonction en lui passant l'état courant (`GraphState`) et fusionnera ce qu'elle retourne dans l'état global via les reducers.

Étape 2 — câblage des arêtes. `add_edge(A, B)` signifie : quand le nœud A termine, passer au nœud B. C'est le pipeline séquentiel principal. `add_conditional_edges(A, fonction, cibles)` signifie : quand A termine, appeler `fonction(state)` pour décider vers quel nœud aller. C'est utilisé à deux endroits : pour le fan-out vers les éva-

luateurs/vérificateurs via `Send()`, et pour le routage conditionnel après injection RAG (vers A7 ou directement vers le rapport).

Étape 3 — compilation. `graph.compile()` transforme la description déclarative en un exécutable. Le `checkpointer=MemorySaver()` active la persistance d'état : après chaque nœud, LangGraph sauvegarde un snapshot de `GraphState`. C'est ce qui permet de reprendre le pipeline après le HITL exactement là où il s'est arrêté. `interrupt_before=["recruteur"]` indique que l'exécution doit se suspendre juste avant le nœud `recruteur`, quelle que soit la vitesse du pipeline.

Listing 5.3 – src/graph.py : topologie complète du graphe

```
graph = StateGraph(GraphState)

# -- Noeuds --
graph.add_node("orchestrateur", orchestrateur_node)   # A1
graph.add_node("analyste",       analyste_node)        # A2
graph.add_node("chercheur_stratege",   stratege_node)  # A3a
graph.add_node("chercheur_collecteur", collecteur_node)# A3b
graph.add_node("chercheur_filtre",     filtre_node)    # A3c
graph.add_node("deduplicateur",  deduplicateur_node)   # A6
graph.add_node("evaluateur",     evaluateur_node)      # A4 (x N)
graph.add_node("reduce_scores",  reduce_scores_node)
graph.add_node("verificateur",   verificateur_node)    # A5 (x N)
graph.add_node("reduce_validations", reduce_validations_node)
graph.add_node("injection_rag",  injection_rag_node)
graph.add_node("recruteur",      recruteur_node)       # A7
graph.add_node("rapport",        rapport_node)         # A1
graph.add_node("persistance",    persistance_node)     # A8

# -- Aretes sequentielles --
graph.add_edge(START, "orchestrateur")
graph.add_edge("orchestrateur",        "analyste")
graph.add_edge("analyste",             "chercheur_stratege")
graph.add_edge("chercheur_stratege",   "chercheur_collecteur")
graph.add_edge("chercheur_collecteur", "chercheur_filtre")
graph.add_edge("chercheur_filtre",     "deduplicateur")
graph.add_edge("evaluateur",           "reduce_scores")
graph.add_edge("verificateur",         "reduce_validations")
graph.add_edge("reduce_validations",   "injection_rag")
graph.add_edge("recruteur",  "rapport")
graph.add_edge("rapport",    "persistance")
graph.add_edge("persistance", END)

# -- Fan-out : Send() -> N instances paralleles --
graph.add_conditional_edges("deduplicateur",   route_to_evaluateurs
    ,
```

```
                                ["evaluateur",    "reduce_scores"])
graph.add_conditional_edges("reduce_scores",
    route_to_verificateurs,
                                ["verificateur",  "reduce_validations"
                                    ])

# -- Routage conditionnel apres injection RAG --
graph.add_conditional_edges("injection_rag",
    route_apres_verification,
                                {"recruteur": "recruteur", "rapport":
                                    "rapport"})

# -- Compilation avec checkpointer et HITL --
app = graph.compile(
    checkpointer=MemorySaver(),
    interrupt_before=["recruteur"]
)
```

L'invocation du graphe se fait en trois phases depuis l'API FastAPI.

Listing 5.4 – src/api.py : invocation du graphe avec streaming et reprise HITL

```
config = {"configurable": {"thread_id": run_id}}

# Phase 1 : execution jusqu'a l'interrupt ou la fin
for event in app_graph.stream(
    {"fiche_poste": fiche_poste},
    config=config,
    stream_mode="updates",          # un evenement par noeud
        complete
):
    _push_event(run_id, "node_completed", {"node": node_name})

# Phase 2 : detection du point d'arret HITL
state = app_graph.get_state(config)
if state.next:                      # pipeline suspendu avant "
    recruteur"
    hitl_event.wait(timeout=600)    # bloque jusqu'a decision
        humaine

# Phase 3 : reprise apres decision (update_state modifie la liste)
app_graph.update_state(config, {"candidats_valides":
    candidats_modifies})
for event in app_graph.stream(None, config=config, stream_mode="
    updates"):
    ...                             # reprend exactement la ou ca s'
        est arrete
```

Phase 1 — exécution streamée. `app_graph.stream()` démarre le pipeline et retourne un événement à chaque fois qu'un nœud se termine. Le `thread_id` dans le `config` identifie le run : LangGraph l'utilise pour stocker les snapshots dans le `MemorySaver`. Chaque événement est immédiatement transmis à l'interface via la queue SSE.

Phase 2 — détection du HITL. Quand le pipeline atteint le point d'arrêt, `stream()` retourne sans aller plus loin. `get_state(config)` permet de lire l'état courant : si `state.next` est non vide, le pipeline est suspendu et attend une décision. Le thread API se bloque alors sur un `threading.Event` jusqu'à ce que le recruteur réponde.

Phase 3 — reprise. `update_state()` modifie directement le `GraphState` stocké dans le checkpointer (par exemple pour exclure un candidat). Puis `stream(None)` reprend l'exécution depuis le point d'arrêt, sans rejouer les nœuds déjà exécutés. C'est la valeur clé du checkpointing LangGraph.

Le `Send()` lance N instances d'A4 en parallèle plutôt qu'une boucle séquentielle. En pratique, évaluer 10 candidats passe de 30–40 s à environ 3–4 s. On a limité à 10 instances simultanées (`MAX_PROFILS_PARALLELES`) pour ne pas dépasser le quota Groq, avec une logique de retry exponentiel en cas d'erreur 429.

Pour le HITL, on a choisi `interrupt_before=["recruteur"]` plutôt qu'un simple `input()` parce que cette option suspend l'état complet du graphe. Le recruteur peut modifier la liste des candidats et reprendre exactement là où le pipeline s'était arrêté, sans perdre ce qui a déjà été calculé.

5.5 Rôle du LLM dans le système

5.5.1 Principe : LLM pour le cognitif, déterministe pour le structurel

Un des apprentissages centraux de ce projet est que le LLM ne doit pas tout faire. Invoquer Groq a un coût : temps d'inférence, tokens consommés sur le quota du *free tier*, et surtout non-déterminisme — deux appels identiques peuvent donner des réponses différentes. Il est donc contre-productif d'utiliser le LLM pour des tâches que du code Python ordinaire peut accomplir de façon instantanée, auditables et reproductibles.

La règle de conception appliquée dans tout le prototype est la suivante :

— **LLM** pour les tâches *cognitives et sémantiques* : comprendre du langage naturel, extraire de l'information non structurée, évaluer une adéquation floue, générer du texte personnalisé.

— **Code déterministe** pour tout le reste : collecte HTTP, filtrage par règles, déduplication, persistance, routage par seuils numériques.

Cette frontière est une instanciation directe du paradigme **neuro-symbolique** identifié dans l'état de l'art : les LLM apportent la flexibilité cognitive là où elle est indispensable ; les composants déterministes garantissent l'auditabilité là où la décision doit être binaire et traçable.

5.5.2 Cartographie des appels LLM par agent

TABLE 5.1 – Agents LLM vs agents déterministes dans le pipeline

Agent	LLM ?	Rôle et justification
A1 — Orchestr.	×	Formate le rapport en Python pur (f-string + tri)
A2 — Analyste	✓	Parse la fiche de poste (texte libre) → JSON structuré
A3a — Stratège	✓	Génère un plan de requêtes adapté par source (DDG, GitHub, SO)
A3b — Collecteur	×	Collecte HTTP (DuckDuckGo, GitHub API, Stack Exchange)
A3c — Filtre	×	Filtrage par règles : 200+ patterns URL et texte
A6 — Dédup.	×	Similarité de noms via `SequenceMatcher`
A4 — Évaluateur	✓	Scoring multicritères de chaque candidat (×N parallèles)
A5 — Vérificateur	✓	Contre-évaluation + 5 guards déterministes post-LLM
Injection RAG	×	Requête ChromaDB par similarité cosinus
A7 — Recruteur	✓	Rédige les messages de contact personnalisés
A8 — Persistance	×	Écriture dans ChromaDB

Concrètement, le LLM remplace dans notre code ce qu'aucune règle déterministe ne peut faire de façon générale :

A2 — Parser une fiche de poste.

Une fiche peut exprimer la même exigence de dizaines de façons différentes : *"5*

ans d'XP", *"confirmé"*, *"senior"*, *"minimum bac+5 avec pratique CI/CD"*. Un regex ne peut pas couvrir toutes les formulations. A2 envoie le texte brut au LLM et reçoit un JSON normalisé (`hard_skills`, `niveau_experience`, `localisations`...) que tous les agents suivants lisent de façon fiable et typée.

A3a — Générer des requêtes adaptées à chaque source.
DDG, GitHub et Stack Overflow ont chacun leur syntaxe de recherche : `site:linkedin.com/in` pour DDG, `language:python followers:>5` pour GitHub, tags techniques pour SO. A3a demande au LLM de produire un plan différencié par source à partir du profil structuré produit par A2. Un générateur de requêtes statique produirait les mêmes mots-clés pour toutes les sources, manquant les opérateurs propres à chacune.

A4 — Scorer l'adéquation candidat/poste.
Le profil brut d'un candidat est un texte hétérogène (bio GitHub, snippet DDG, page Stack Overflow). Calculer un score multicritères (`hard_skills`, `soft_skills`, `experience`, `culture_fit`) sur ce type de texte est une tâche sémantique : les compétences ne sont pas toujours listées explicitement, il faut les inférer du contexte. C'est la tâche centrale du LLM dans notre système.

A5 — Valider et affiner le score d'A4.
A5 reçoit le score d'A4 (`score_global_A4`) et l'utilise comme point de départ. Le LLM contrôle l'adéquation réelle du candidat et peut ajuster ce score à la hausse ou à la baisse selon ce qu'il observe dans le texte brut. Les cinq guards déterministes s'appliquent ensuite pour corriger ou invalider le résultat si nécessaire (page non-candidat, compétences insuffisamment prouvées, niveau incompatible).

A7 — Rédiger des messages de contact personnalisés.
Un message de contact efficace mentionne des éléments précis du profil du candidat. Ce niveau de personnalisation est impossible avec un template fixe : la mention d'un projet GitHub spécifique, d'une réponse Stack Overflow notable ou d'un titre de poste actuel nécessite que le LLM lise et synthétise le texte brut du candidat.

Patron d'invocation commun. Les cinq agents LLM suivent tous le même patron d'appel. Ce qui diffère d'un agent à l'autre, c'est uniquement le prompt système, le contenu du message humain, et la température :

Listing 5.5 – Patron d'invocation LLM commun aux 5 agents (src/agents/*.py)

```
# Identique dans A2, A3a, A4, A5 et A7 :
llm = get_llm(temperature=0)      # 0 = deterministe (A2, A3a, A4, A5)
```

```
                                    # 0.7 = creatif pour A7 (messages
                                        personnalises)
messages = [
    SystemMessage(content=PROMPT_SYSTEME),    # role + contraintes
        de l'agent
    HumanMessage(content=contexte),           # donnees specifiques
        au run
]
response = llm.invoke(messages)
# Post-traitement : parse JSON (A2, A3a, A4, A5) ou texte libre (A7
    )
```

Ce que chaque agent met dans `contexte` :

TABLE 5.2 – Entrée et sortie LLM par agent

Agent	HumanMessage (entrée)	Sortie parsée
A2	Texte brut de la fiche de poste	JSON : `hard_skills`, `niveau_experience`, `localisations`...
A3a	Profil structuré produit par A2 (hard/soft skills, niveau, lieu)	JSON : requêtes par source (DDG, LinkedIn, GitHub, SO)
A4	Profil requis + texte brut du candidat + contexte RAG	JSON : `score_global`, `scores_detail`, `resume`
A5	Profil requis + texte brut + score A4 (`score_global_A4`)	JSON : `score_final`, `statut`, `remarques`
A7	Fiche de poste + liste des candidats validés (JSON)	Liste JSON de messages (`objet`, `message`, `canal`)

5 agents sur 10 appellent le LLM, les 5 autres sont entièrement déterministes. La moitié du pipeline ne dépend donc ni du quota Groq ni de la variabilité du modèle : filtrage, déduplication, collecte et persistance produisent toujours le même résultat pour les mêmes entrées. Le cache RAG dans A4 pousse cette logique plus loin : si un candidat a déjà été évalué pour un poste similaire, l'appel LLM est court-circuité et le score est réutilisé directement depuis ChromaDB.

5.5.3 Choix du fournisseur : pourquoi Groq et pas Ollama ou un autre

En début de projet, nous avons testé plusieurs options avant d'arrêter notre choix.

Les **modèles locaux via Ollama** (Mistral, Llama 2 et d'autres variantes) ont été testés en premier car ils ne nécessitent aucune clé API et ne transmettent aucune donnée à un tiers — ce qui est idéal du point de vue RGPD. Deux problèmes les ont écartés pour le prototype :

— **Fiabilité des sorties JSON.** Malgré un prompt exigeant un JSON pur sans balises Markdown, Mistral en particulier enveloppait régulièrement sa réponse dans des blocs "‘`json...`"‘ ou oubliait des clés. Le post-traitement de récupération fonctionnait dans 70–80% des cas mais introduisait de la fragilité.

— **Latence.** Sur machine de développement, un appel Mistral local prenait > 10 s. Avec dix instances d'A4 en parallèle, le pipeline aurait dépassé 100 s rien que pour le scoring — inacceptable pour une démonstration.

Groq s'est imposé pour trois raisons concrètes :

1. **Latence LPU.** Groq utilise des processeurs dédiés à l'inférence (Language Processing Unit). En pratique, la quasi-totalité des prompts revenaient en moins de 2 s, contre plus de 10 s en local.
2. **Free tier opérationnel.** La limite de 100 000 tokens/jour est suffisante pour 3 à 5 runs complets — assez pour le développement, les tests et les démonstrations.
3. **Compatibilité API OpenAI.** L'interface Groq est identique à celle d'OpenAI, ce qui permet d'utiliser `init_chat_model` de LangChain avec `model_provider="groq"` ou `"ollama"` sans modifier une ligne dans les agents. Le changement de provider est une variable d'environnement, pas une modification de code.

5.5.4 Implémentation centralisée : `get_llm()` et gestion du quota

L'accès au LLM est centralisé dans une seule fonction dans `src/config.py`. Aucun agent n'instancie directement `ChatGroq` ou `ChatOllama` : ils appellent tous `get_llm()`, ce qui garantit que le provider peut être changé en un seul endroit.

Listing 5.6 – Fournisseur LLM centralisé avec retry (src/config.py)

```
LLM_PROVIDER = os.getenv("LLM_PROVIDER", "groq")
if "LLM_MODEL" in os.environ:           # forcer le modele
    independamment du provider
    LLM_MODEL = os.environ["LLM_MODEL"]
elif LLM_PROVIDER == "groq":
    LLM_MODEL = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")
else:
    LLM_MODEL = "mistral"               # fallback Ollama local

def get_llm(temperature: float = 0):
```

```
    """Retourne le LLM configure avec retry exponentiel sur rate limit."""
    if LLM_PROVIDER == "groq" and not GROQ_API_KEY:
        raise RuntimeError("GROQ_API_KEY manquante. Copiez .env.example -> .env")
    llm = init_chat_model(LLM_MODEL,
                          model_provider=LLM_PROVIDER,
                          temperature=temperature)
    # 6 tentatives avec backoff aleatoire si erreur 429
    return llm.with_retry(stop_after_attempt=6,
        wait_exponential_jitter=True)
```

Le `.with_retry()` intégré à LangChain gère les erreurs 429 (quota dépassé) avec un backoff exponentiel aléatoire (*jitter*). Mais lors des évaluations parallèles via `Send()`, dix instances d'A4 envoient leurs requêtes simultanément : les premières passent, les suivantes reçoivent toutes un 429 au même moment. Le `with_retry` seul ne suffit pas car toutes les instances retentraient au même instant. A4 ajoute donc une seconde couche avec un délai *différent* par instance :

Listing 5.7 – Double couche de protection contre le rate limit (src/agents/evaluateur.py)

```
llm = get_llm(temperature=0)    # 1re couche : with_retry(6) de LangChain

# 2e couche : backoff manuel pour espacer les retentatives concurrentes
for attempt in range(5):
    try:
        response = llm.invoke(messages)
        break
    except Exception as e:
        if "429" in str(e) or "too many" in str(e).lower():
            wait = (2 ** attempt) + random.uniform(0, 1)  # jitter aleatoire
            log.warning("Rate limit %s, retry %d/5 dans %.1fs",
                        candidat['nom'], attempt + 1, wait)
            time.sleep(wait)
        else:
            raise
```

Le backoff 2^n + jitter espace les retentatives dans le temps et les rend asynchrones entre instances : deux instances qui frappent le quota au même instant ne retentent pas au même moment. En pratique, sur les 3 runs documentés, aucun échec définitif n'a été observé malgré les 10 évaluations parallèles.

5.6 Les agents et leurs choix de conception

5.6.1 A1 : Orchestrateur

A1 est le point d'entrée et de sortie du pipeline. Il initialise l'état global, coordonne l'exécution des agents spécialisés via le flux du graphe, et génère lui-même le rapport final en Markdown (tri des candidats, insertion des scores, formatage). Ce double rôle, coordination et production du livrable final, est intentionnel et correspond au **pattern Superviseur** des SMA classiques.

Listing 5.8 – Structure du rapport Markdown généré par A1 (src/agents/orchestrateur.py)

```
rapport = f"""# Rapport final de recrutement

## Resume du poste
- Hard skills : {hard_skills}
- Niveau d'experience attendu : {niveau}

## Statistiques de recherche
- Resultats bruts collectes : {n_hits_bruts}
- Profils apres deduplication : {n_dedup}
- Candidats valides : {len(candidats_valides)}

## Candidats valides a contacter
{_table_candidats(candidats_valides, profils_par_id)}

## Messages generes
{messages_section}

## Metriques d'execution
{metriques_resume}
"""
```

5.6.2 A2 : Analyste de poste

La fiche de poste est fournie en texte libre par l'utilisateur. Un traitement par règles fixes (regex, mots-clés) serait insuffisant face à la variété des formulations. A2 appelle `get_llm()` (voir section 5.5) pour transformer ce texte en JSON structuré (`hard_skills`, `soft_skills`, `niveau_experience`, `localisations`, `remote`...), ce qui permet à tous les agents suivants de raisonner sur des données typées plutôt que sur du texte brut. Un enrichissement déterministe post-LLM (regex pour le niveau d'expérience) corrige les cas où le modèle est imprécis.

Listing 5.9 – Extrait du prompt A2 (src/prompts.py)

```
ANALYSTE_SYSTEM = """
Extrais un JSON avec ces cles :
- hard_skills, soft_skills, niveau_experience, localisations,
   remote, ...
ATTENTION : AWS, GCP, Docker ne sont PAS des localisations.
Reponds UNIQUEMENT avec le JSON, sans markdown ni backticks.
"""
```

5.6.3 A3 : Pipeline de collecte (trois sous-agents)

A3 est décomposé en trois sous-agents plutôt qu'un seul, pour séparer trois responsabilités de nature différente : la *stratégie* (cognitif, LLM), la *collecte* (logistique, déterministe) et le *filtrage* (algorithmique, sans LLM). Cette décomposition permet de tester, modifier et déboguer chaque couche indépendamment, et d'éviter de consommer des tokens Groq pour des tâches purement mécaniques.

A3a, Stratège. Génère un plan de recherche adapté par source (DDG, LinkedIn, GitHub, Stack Overflow) via LLM. Les opérateurs de recherche (`site:linkedin.com/in`, `language:python followers:>5`) sont ajoutés en post-traitement déterministe pour garantir leur présence même si le LLM les omet.

A3b, Collecteur. Exécute le plan sur trois sources gratuites : DuckDuckGo, GitHub Search Users API (60 req/h sans token), Stack Exchange API. Délai de 1,5 s entre appels DDG pour éviter le rate-limiting. Aucun LLM ici, pure logistique.

DuckDuckGo est un moteur de recherche grand public axé sur la confidentialité. Contrairement à Google, il expose une interface de recherche textuelle accessible sans clé API via la bibliothèque Python `ddgs`. Son rôle dans notre système est d'indexer le web public : on lui soumet des requêtes ciblées sur des profils LinkedIn (`site:linkedin.com/in`), GitHub (`site:github.com`) ou des CV en ligne, et il retourne pour chaque résultat un titre, une URL et un court extrait (snippet). La limite est que DuckDuckGo ne donne pas accès au contenu complet des pages — seulement au snippet — ce qui explique la qualité limitée des profils LinkedIn récupérés par cette voie. Des exclusions automatiques sont ajoutées à chaque requête (`-"offre d'emploi" -"postuler" -"CDI à pourvoir"`) pour éviter que les résultats soient dominés par des annonces d'emploi plutôt que des profils de candidats.

GitHub est la plateforme d'hébergement de code la plus utilisée au monde. Les développeurs y publient leurs projets open-source, contributions et portfolios techniques. GitHub expose une API officielle REST qui permet, entre autres, de rechercher des utilisateurs par compétences, localisation et activité (`language:python`, `location:Paris`,

`followers:>5`). Pour chaque utilisateur trouvé, A3b récupère son profil complet — bio, localisation, entreprise, nombre de dépôts publics, followers — via un second appel à `/users/{login}`. L'avantage est que les données sont structurées et fiables : un profil GitHub reflète l'activité réelle d'un développeur, pas un CV auto-déclaré. Sans token, la limite est de 60 requêtes par heure, suffisant pour un run.

Stack Overflow est la principale communauté mondiale de questions-réponses pour les développeurs. Les utilisateurs y posent et répondent à des questions techniques, et accumulent une *réputation* proportionnelle à la qualité de leurs contributions. Stack Exchange API est l'API publique qui expose ces données. A3b l'utilise pour récupérer les utilisateurs les plus actifs sur des tags correspondant aux compétences recherchées (`python`, `machine-learning`, `fastapi`), triés par réputation. Une forte réputation sur un tag signifie que la personne répond régulièrement et correctement à des questions sur ce sujet — c'est un signal de compétence technique vérifiable et objectif. Sans clé API, la limite est de 300 requêtes par jour ; une clé gratuite sur `stackapps.com` monte à 10 000.

A3c, Filtre anti-bruit. Quatre passes algorithmiques sans LLM : pré-filtre URL (`_NOISE_DOMAINS` et `_NOISE_URL_PATHS`), scraping HTTP avec repli snippet DDG en cas d'échec, détection de pages agrégées (3+ URL `/in/` dans la page), signal bruit (50+ patterns offres d'emploi, le profil n'est pas supprimé mais reçoit un préfixe `[Signal A3c]` pour alerter A4 et A5). La justification du choix algorithmique est détaillée dans la section suivante.

5.6.4 A6 : Déduplicateur

Le même candidat peut apparaître sur plusieurs sources avec des URLs différentes. Une déduplication par URL seule serait insuffisante. A6 fusionne par URL identique ou par similarité de nom ($> 85\%$ via `SequenceMatcher`). Le seuil de 85% a été retenu après tests : en dessous, des homonymes sont fusionnés par erreur ; au-dessus, des variantes légitimes du même nom (avec ou sans accent, initiale) sont manquées. Les profils fusionnés voient leurs textes concaténés, ce qui maximise l'information disponible pour A4.

Listing 5.10 – Détection et fusion de doublons (src/agents/deduplicateur.py)

```
def _are_duplicates(p1, p2) -> bool:
    # Meme URL = meme personne
    if p1.get("url") and p2.get("url") and p1["url"] == p2["url"]:
        return True
    # Similarite de nom > 85%
    ratio = SequenceMatcher(
        None, p1["nom"].lower().strip(), p2["nom"].lower().strip()
```

```
    ).ratio()
    return ratio > 0.85

# Fusion : concatener les textes des deux sources
merged["profil_brut"] = (
    merged["profil_brut"] + "\n\n--- Source: "
    + p2["source"] + " ---\n" + p2["profil_brut"]
)
```

5.6.5 A4 : Évaluateur ($\times N$ parallèles)

A4 évalue chaque candidat indépendamment avec un barème strict dans le prompt : 85–100 (excellent), 70–84 (bon), 50–69 (intéressant), 30–49 (partiel), 0–29 (hors sujet). Le barème explicite est nécessaire car sans contrainte, les LLM tendent à concentrer les scores dans une plage étroite. Si une URL est disponible pour le candidat, A4 consulte d'abord le cache RAG : si un score récent existe pour cette URL sur une fiche similaire, il est réutilisé sans appeler le LLM.

Listing 5.11 – Cache RAG dans A4 (src/agents/evaluateur.py)

```
cache = get_memoire().get_score_cache(url, fiche_poste)
if cache is not None:
    return {"candidats_scores": [CandidatScore(score_global=cache["
        score"])]}
```

5.6.6 A5 : Vérificateur ($\times N$ parallèles)

A4 et A5 sont séparés car l'évaluation LLM est probabiliste : un même prompt peut produire des scores différents, et les biais du modèle peuvent favoriser certains profils. Un second agent indépendant, le **pattern validation pair-à-pair** des SMA classiques, applique un regard critique. A5 invoque d'abord Groq pour obtenir un score et un statut, puis applique des garde-fous déterministes qui peuvent corriger le résultat à la baisse si des incohérences sont détectées.

Guards déterministes (cinq niveaux successifs) :

Listing 5.12 – Guards A5 (src/agents/verificateur.py)

```
# Guard 1 : page non-candidat (offre, formation, agregateur)
if _profil_non_candidat(nom, profil_brut, url):
    score_final = min(score_final, 20.0); statut = "invalide"

# Guard 2 : page ne decrivant pas clairement une personne
elif statut == "valide" and not _profil_personne_probable(
        nom, profil_brut, source, url):
```

```
    score_final = min(score_final, 45.0); statut = "invalide"

# Guard 3 : competences insuffisamment prouvees dans le texte
elif statut == "valide":
    ok, raison = _adequation_minimale(profil_requis, nom,
        profil_brut)
    if not ok:
        score_final = min(score_final, 45.0); statut = "invalide"

# Guard 4 : niveau d'experience incompatible (senior pour un poste
    junior)
incompatible, raison = _experience_incompatible(profil_brut,
    niveau_requis)
if incompatible:
    score_final = min(score_final, 45.0); statut = "invalide"

# Guard 5 : upgrade LinkedIn -- profil /in/ avec titre non-tech
    exclu
_titre_non_tech = any(t in nom.lower() for t in (
    "banquier", "banker", "retraite", "immobilier", "notaire",
    "avocat", "juriste", "commercial", ...))  # 16 termes au total
if (statut == "douteux" and score_a4 >= SCORE_SEUIL_CONTACT
        and "/in/" in url.lower() and not _titre_non_tech):
    statut = "valide"   # confiance au score A4 malgre scraping
        bloque
```

5.6.7 Nœud injection RAG

Après les validations A5, un nœud dédié récupère les candidats validés lors de runs précédents pour une fiche similaire (cosine ≥ 0.75, ≤ 30 jours) et les injecte dans les résultats courants. Ce nœud est silencieux au premier run (base vide) et devient utile à partir du second. Son intérêt est de ne pas réévaluer un candidat déjà jugé excellent pour un poste analogue.

5.6.8 A7 : Recruteur

A7 rédige des messages de contact personnalisés (max 150 mots). Le routage vers A7 est conditionnel : si le meilleur score dépasse 75 (mode absolu), ou si des candidats viables (≥ 40) existent (mode relatif top-3). Ce double seuil évite d'envoyer des messages pour des profils médiocres tout en garantissant qu'un recrutement difficile (aucun excellent) aboutit quand même à des candidatures raisonnables.

Limite du prototype. Dans l'état actuel, A7 rédige les messages mais ne les envoie pas : ils sont simplement stockés dans le rapport final et affichés dans l'interface. Il n'y

a aucune intégration avec un canal réel (email, API LinkedIn, Slack). Dans un projet en production, l'amélioration naturelle serait d'ajouter un nœud d'envoi après A7 : une fois les profils validés et les messages rédigés, l'utilisateur recevrait une notification réelle (email ou alerte UI) avec les messages prêts à envoyer, qu'il pourrait approuver et déclencher en un clic.

Human-in-the-Loop avant A7. Le HITL s'intercale juste avant A7 à deux conditions : la case *Activer HITL* doit être cochée à la page Accueil avant de lancer le run — sans cela, `with_interrupt=False` et le pipeline s'exécute entièrement en automatique — et le routage doit atteindre effectivement A7, c'est-à-dire qu'il faut au moins un candidat avec un score ≥ 75 ou trois candidats viables ≥ 40. Si aucun candidat ne passe ces seuils, le pipeline saute A7 et va directement au rapport — le HITL n'apparaît alors jamais, même avec la case cochée.

Quand il s'active, le recruteur dispose de trois actions : **approve** (A7 traite tous les candidats validés), **skip** (A7 est sauté, aucun message rédigé), ou **edit** (modification du JSON `candidats_valides` avant traitement). Les décisions de rejet sont ensuite transmises à A8, qui écrase le statut en `invalide` dans ChromaDB : la mémoire RAG intègre ainsi les corrections humaines pour les runs futurs.

Listing 5.13 – Prompt A7 (src/prompts.py)

```
RECRUTEUR_SYSTEM = """Tu es un recruteur expert. Ta mission est de rediger
des messages de premier contact personnalises pour les meilleurs candidats.

Pour chaque candidat valide, redige un message qui :
1. Mentionne specifiquement ce qui dans leur profil a retenu l'attention
2. Presente brievement le poste et l'entreprise
3. Propose un echange (appel, visio, cafe)
4. Reste concis (max 150 mots)

Pour chaque candidat, produis un JSON :
{
  "candidat_id": "...",
  "nom": "...",
  "objet": "Objet du message",
  "message": "Corps du message",
  "canal": "linkedin | email"
}
Reponds UNIQUEMENT avec la liste JSON."""
```

Listing 5.14 – Routage conditionnel (src/graph.py)

```python
if meilleur_score >= SCORE_SEUIL_CONTACT:  # 75
    return "recruteur"
viables = [c for c in candidats if c["score_final"] >= SCORE_SEUIL_VIABLE]  # 40
if viables:
    return "recruteur"   # top-3 relatif
return "rapport"         # aucun candidat viable
```

5.6.9 A8 : Persistance RAG

A8 ferme la boucle mémoire. Sa stratégie de persistance est calibrée : `valide` → cache de score pour éviter de réévaluer ce candidat au prochain run ; `invalide` → blacklist inter-runs pour ne plus scraper cette URL ; `douteux` → non persisté car trop incertain pour servir de référence. Le rejet HITL écrase un statut valide précédent, la décision humaine a toujours le dernier mot.

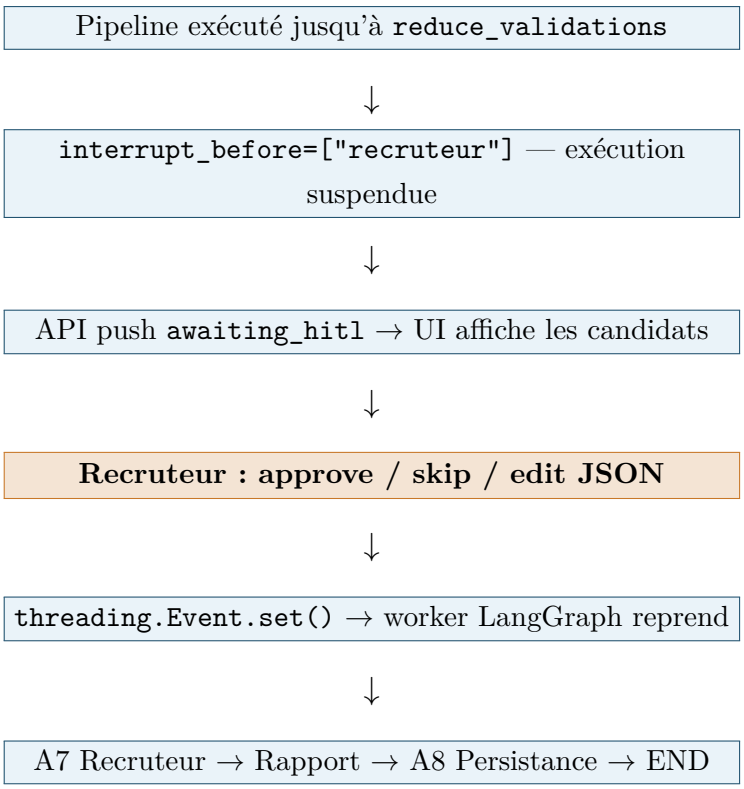


FIGURE 5.2 – Séquence Human-in-the-Loop : suspension avant A7 et reprise après décision humaine.

5.7 Mémoire RAG

ChromaDB [17] local (`./data/chromadb/`) + SentenceTransformers (`all-MiniLM-L6-v2`). Deux collections liées par `fiche_id` (hash SHA1 de la fiche) :

TABLE 5.3 – Collections ChromaDB

Collection	Document	Métadonnées
`fiches_poste`	Texte fiche (3000 chars)	longueur
`candidats_evalues`	profil_brut (2000 chars)	nom, score, source, fiche_id, statut, url, last_seen

Trois fonctions disponibles :

1. **Blacklist** (A3c) : requête metadata-only, quasi-instantanée. La fonction `est_blackliste()` est implémentée dans le module RAG ; son branchement dans le filtre A3c est prévu comme extension.
2. **Cache de score** (A4) : cosine ≥ 0.85 sur la fiche.
3. **Injection proactive** (nœud dédié) : cosine ≥ 0.75, 30 jours.

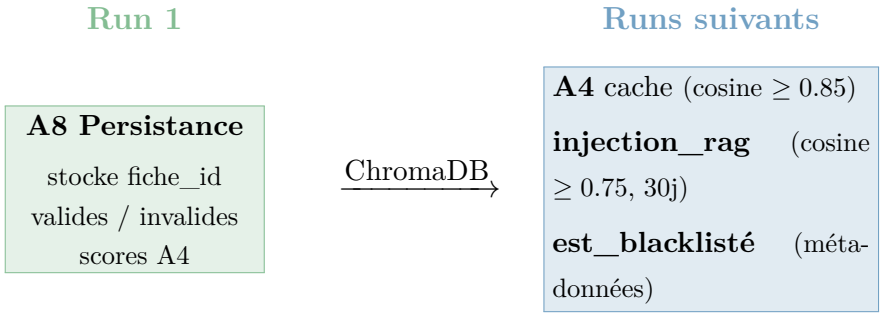


FIGURE 5.3 – Flux de la mémoire RAG calibrée entre deux runs successifs sur une fiche similaire.

5.8 Pipeline de sourcing multi-sources (A3)

L'agent A3 est décomposé en trois sous-agents A3a → A3b → A3c, séparant stratégie, collecte et filtrage.

5.8.1 A3a : Stratège : génération du plan de recherche

À partir du `profil_competences` produit par A2, A3a génère un objet `requetes_recherche` structuré en cinq familles : **queries_generales** (opérateurs `intitle:CV`, `inurl:cv`), **queries_linkedin** (`site:linkedin.com/in`), **queries_github** (`location:Ville language:Pytho followers:>5`), **queries_cv_sites** (`site:malt.fr`, `site:github.io`), et **tags_stackoverflow** (3–5 tags exacts des hard skills).

Un post-traitement déterministe `_enrichir_requetes()` ajoute des termes d'exclusion selon le niveau : pour un profil *alternant*, les termes `-"senior" -"lead" -"manager"` sont ajoutés. Fallback déterministe si le parsing JSON du LLM échoue.

5.8.2 A3b : Collecteur : exécution multi-sources

A3b collecte les profils sur trois sources gratuites sans clé API payante.

DuckDuckGo (source principale). DuckDuckGo est utilisé comme moteur de recherche web via la bibliothèque `ddgs`. Le principe est le suivant : A3a génère des requêtes ciblées (ex. `site:linkedin.com/in "Python" "FastAPI" "Paris"`), A3b les soumet à DDG qui retourne une liste de résultats web, chaque résultat contenant un titre, une URL et un extrait (snippet). Ces URLs sont ensuite scrappées par A3c pour récupérer le contenu réel de la page de profil. DDG est préféré à Google car il ne nécessite aucune clé API et tolère des appels fréquents avec un délai de 1,5 seconde entre requêtes.

GitHub Search Users API. L'API GitHub expose un endpoint de recherche d'utilisateurs (`/search/users`) qui retourne directement des profils développeurs filtrables par localisation, langage et popularité. Contrairement à DDG, les résultats sont déjà des profils de développeurs, pas besoin de scraping supplémentaire. Limites : 60 req/h sans token, 5000 req/h avec `GITHUB_TOKEN` (gratuit).

Stack Exchange API. Recherche d'utilisateurs par tags techniques (`/users?tagged=python&fast`
Retourne des profils avec réputation, badges et spécialisations. Limite : 300 req/jour sans clé, 10 000 avec une clé gratuite (`STACKOVERFLOW_KEY`).

Chaque résultat des trois sources est normalisé dans un format unifié `{title, url, body, source}`, puis dédupliqué par URL avant d'être transmis à A3c :

Listing 5.15 – Normalisation et déduplication (A3b)

```
seen: set[str] = set()
deduped: list[dict] = []
for h in all_hits:
    url = h.get("url", "")
    if not url or url in seen:
        continue
    seen.add(url)
    deduped.append(h)
```

Processus de scraping (A3c). Une fois les URLs collectées par A3b, A3c scrape chaque page via `requests` + `BeautifulSoup4` pour en extraire le texte brut (max

3000 caractères). C'est ce texte qui constitue le `profil_brut` transmis à A4 et A5 pour l'évaluation. En cas d'échec HTTP (LinkedIn bloque quasi-systématiquement), le snippet DDG est utilisé comme substitut partiel.

5.8.3 A3c : Filtre anti-bruit : quatre étapes algorithmiques

A3c applique quatre étapes *sans appel LLM* : (1) pré-filtre URL sur `_NOISE_DOMAINS` et `_NOISE_URL_PATHS` ; (2) scraping avec repli sur snippet DDG en cas d'échec HTTP ; (3) détection de pages agrégées (3+ URL `/in/` dans la page) ; (4) signal bruit : si `_is_noise_text()` détecte un mot-clé, le profil reçoit le préfixe `[Signal A3c]` pour alerter A4 et A5.

Un filtrage par LLM représenterait 200–400 appels supplémentaires par exécution. Les règles algorithmiques s'exécutent en moins d'une milliseconde par hit, sont auditables et restent stables face aux variations du modèle.

5.9 Feature Model

Table 5.4 – Feature model, variabilité architecturale

Feature	Type	Variantes
SMA Recrutement	racine	,
Mode d'exécution	XOR	**Local** (monolithique) | **Microservices** (Docker)
Fournisseur LLM	mand.	**Groq** (`llama-3.3-70b-versatile`) [extensible via `LLM_PROVIDER`]
HITL	XOR	**Activé** | **Désactivé**
Interface	OR	**CLI** | **API REST** | **UI Web**
Mémoire RAG	AND	**Blacklist** + **Cache** + **Injection**

XOR = alternatif exclusif OR = un ou plusieurs AND = tous obligatoires

5.10 API Gateway

L'API Gateway FastAPI [18] (`src/api.py`) est le seul point d'entrée public du système. Elle expose dix endpoints REST :

TABLE 5.5 – Endpoints de l'API Gateway

Endpoint	Rôle
`POST /recruter`	Lance un run, retourne un `run_id`
`GET /rapport/{id}`	Rapport et état complet d'un run
`GET /runs/{id}/stream`	Streaming SSE temps réel
`POST /runs/{id}/hitl`	Validation humaine (approve/skip/edit)
`POST /runs/{id}/cancel`	Annulation propre entre deux nœuds
`GET /runs`	Historique de tous les runs
`GET /rag`	État de la mémoire RAG (ChromaDB)
`GET /rag/search`	Recherche de profils similaires
`GET /metrics`	Métriques pipeline (run courant + exports JSON)
`GET /health`	Santé du service

Le streaming SSE (`GET /runs/{id}/stream`) repose sur un pont `queue.Queue` thread-safe entre le worker LangGraph synchrone et le générateur asyncio de FastAPI. Chaque nœud du graphe pousse un événement JSON dans la queue ; le générateur lit et envoie en continu. Cela permet à l'interface d'afficher les logs en temps réel sans polling.

Listing 5.16 – Pont SSE : queue.Queue entre worker et générateur asyncio (src/api.py)

```python
# Worker LangGraph (thread synchrone) -> pousse dans la queue
def _push_event(run_id, event_type, data):
    q = _runs[run_id]["_event_queue"]   # queue.Queue thread-safe
    q.put({"type": event_type, "data": {**data, "ts": datetime.now
        ().isoformat()}})

# Generateur asyncio -> lit la queue et envoie au client SSE
async def event_generator():
    while True:
        try:
            event = q.get_nowait()      # non bloquant
        except queue.Empty:
            await asyncio.sleep(0.1)    # libere la boucle asyncio
            continue
        if event is None:               # sentinel de fin
            yield "event: stream_closed\ndata: {}\n\n"
            break
        yield f"event: {event['type']}\ndata: {json.dumps(event['
            data'])}\n\n"
```

Le mécanisme HITL (`POST /runs/{id}/hitl`) utilise un `threading.Event` : le wor-

ker est bloqué en attente de cet événement après l'interruption LangGraph. Quand l'humain valide ou rejette, l'API lève l'événement et le worker reprend ou abandonne.

5.11 Interface graphique NiceGUI

NiceGUI [19] a été retenu pour l'interface graphique en raison de trois propriétés adaptées au projet : (1) il génère une interface web servie par FastAPI sans JavaScript manuel, ce qui évite une dépendance frontend supplémentaire ; (2) il est strictement cliente d'une API REST, ce qui découple l'UI de LangGraph ; (3) contrairement à Streamlit ou Gradio, il permet une navigation multi-pages avec état partagé, nécessaire pour les six pages du pipeline. L'interface (`src/ui/`) ne connaît pas LangGraph. Elle comporte six pages :

Page 1, Accueil. Formulaire de saisie de la fiche de poste (zone de texte libre), choix d'activation du HITL, bouton de lancement. À la soumission, l'interface appelle `POST /recruter` et redirige vers la page de suivi avec le `run_id` retourné.

Interface : zone de texte libre pour la fiche de poste, case à cocher HITL, bouton « Lancer le pipeline ». La soumission retourne un ***`run_id`*** *et redirige vers la page de suivi.*

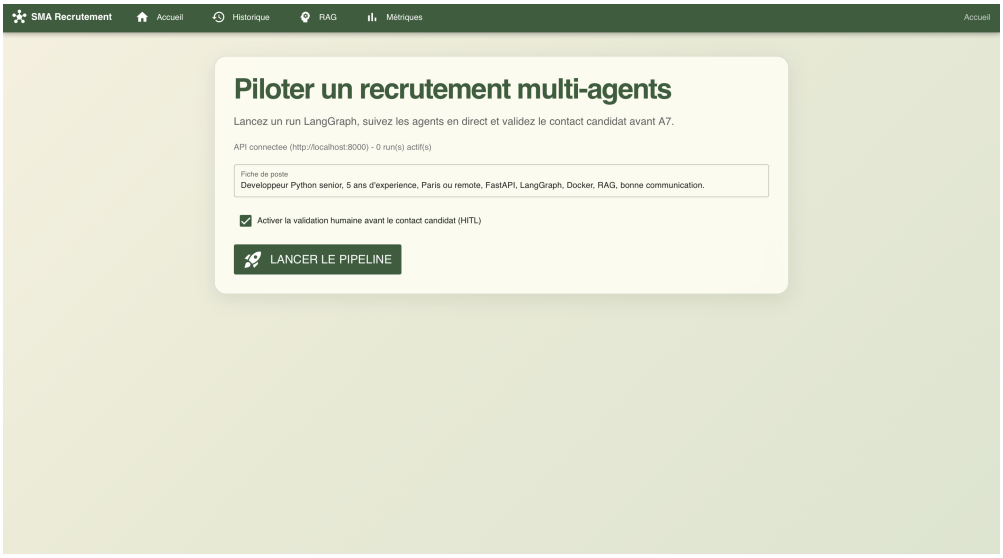


FIGURE 5.4 – Page 1 : formulaire de lancement d'un recrutement.

Page 2, Run en cours. Affiche les logs SSE en temps réel (un événement par nœud LangGraph exécuté) et, quand le pipeline atteint le point d'arrêt HITL, les candidats présélectionnés avec leurs scores. Trois actions sont proposées : approuver, rejeter, ou éditer la liste (JSON éditable).

Interface : flux de logs SSE en temps réel (un événement JSON par nœud), et au point HITL : liste des candidats avec scores et trois boutons (Approuver / Rejeter / Éditer).

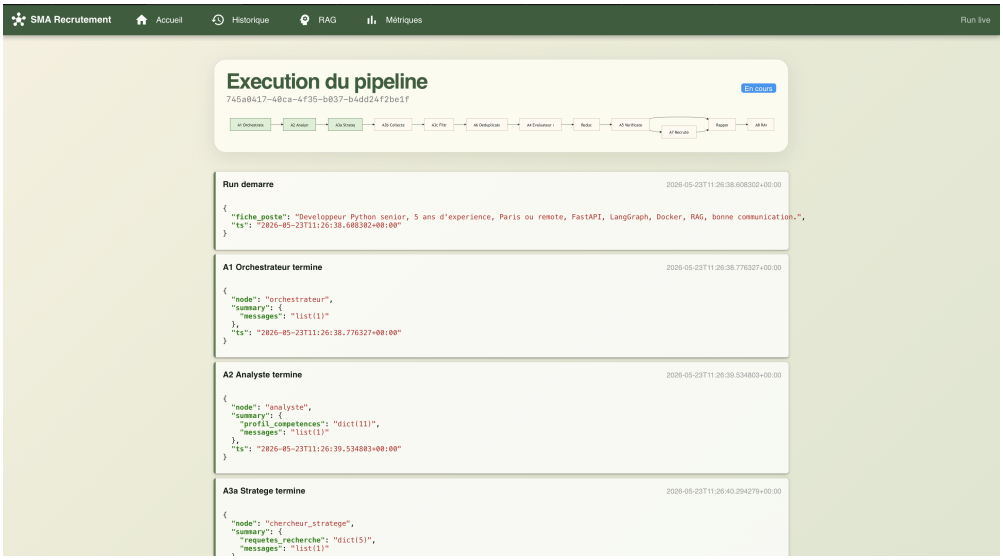


FIGURE 5.5 – Page 2 : point d'arrêt HITL avant A7, décision humaine sur la liste des candidats.

Page 3, Rapport. Affiche le rapport final Markdown généré par A1, la liste des candidats retenus avec scores et statuts. Les messages de contact rédigés par A7 sont inclus dans le rapport Markdown.

Interface : rapport Markdown généré par A1 avec la liste des candidats, leurs scores et les messages de contact rédigés par A7.

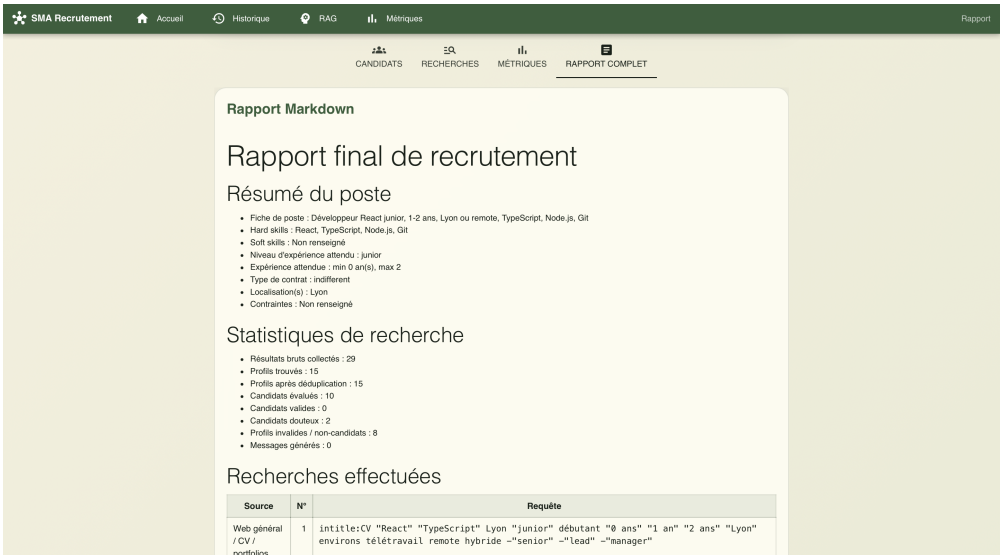


FIGURE 5.6 – Page 3 : rapport final généré par A1 avec les candidats retenus et les messages de contact.

Page 4, Mémoire RAG. Exploration de la base ChromaDB : nombre de fiches indexées, nombre de candidats mémorisés, répertoire de stockage, et moteur de recherche par similarité dans la base vectorielle.

Interface : statistiques ChromaDB (fiches, candidats, répertoire) et moteur de recherche vectoriel.

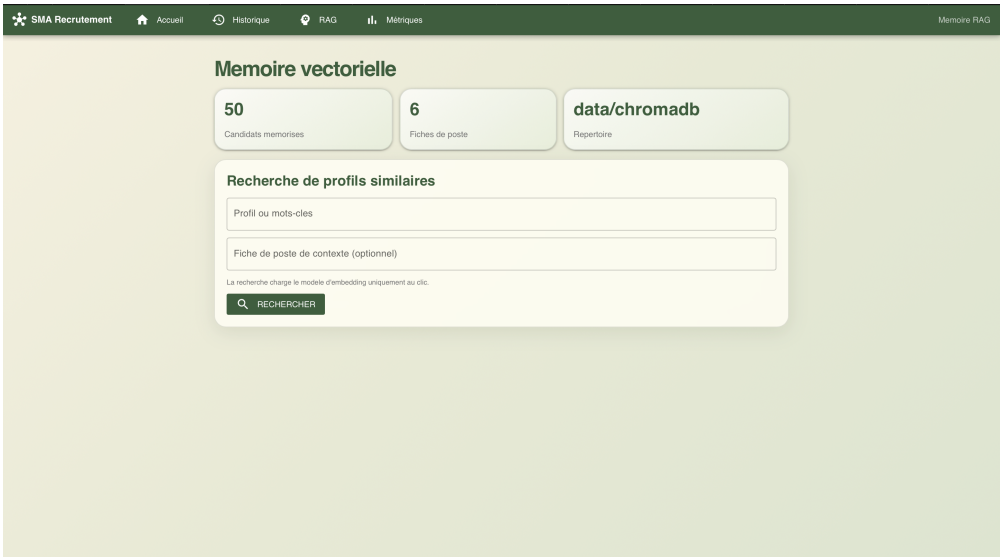


FIGURE 5.7 – Page Mémoire RAG : exploration de la base vectorielle ChromaDB.

Page 5, Métriques. Statistiques du pipeline : nombre de profils collectés, taux de bruit, durée par nœud et par agent. Ces métriques sont exportées dans des fichiers JSON dans le répertoire `logs/` (`metriques_{run_id}.json`). Le comptage des appels Groq et des tokens consommés n'est pas instrumenté dans le prototype actuel.

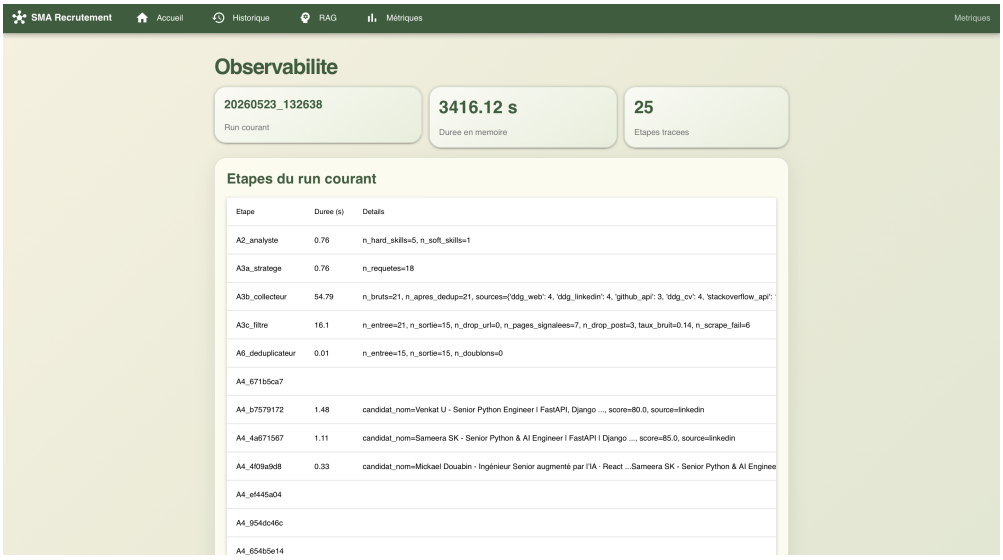


FIGURE 5.8 – Page Métriques : statistiques d'exécution du pipeline par nœud et par agent.

Page 6, Historique. Liste des runs précédents avec leur identifiant, leur statut (terminé, annulé, en attente HITL), et leurs horodatages de début et de fin. Permet de re-consulter un rapport ancien sans relancer le pipeline.

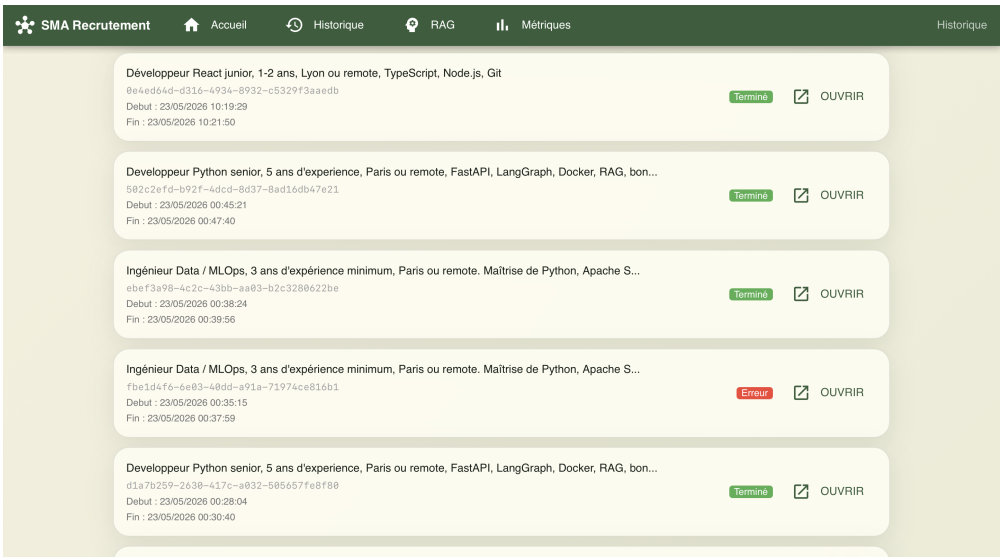


FIGURE 5.9 – Page Historique : liste des runs précédents avec statut et horodatages.

5.12 Deux architectures d'exécution

Le système dispose de deux architectures d'exécution, mais elles ne sont pas équivalentes. **LangGraph monolithique** est l'architecture de référence : c'est elle qui implémente l'intégralité du cahier des charges, notamment le HITL, le checkpointing et le fan-out parallèle. **Microservices Docker** est une architecture secondaire, construite pour montrer que les agents peuvent être déployés dans des conteneurs indépendants sans modifier leur logique métier. Elle ne remplace pas LangGraph : elle en démontre la portabilité. Les deux modes partagent la même API FastAPI, la même interface NiceGUI et le même RAG ChromaDB, mais seul le mode LangGraph respecte toutes les exigences fonctionnelles.

5.12.1 Mode principal : LangGraph monolithique

Principe. LangGraph compile le pipeline en un `StateGraph` typé qui s'exécute dans un unique processus Python. Chaque nœud est une fonction Python ordinaire ; l'état `GraphState` voyage à travers les nœuds et est accessible à tout moment pour inspection ou modification. Le checkpointer `MemorySaver` sauvegarde un snapshot d'état après chaque nœud.

Ce que LangGraph apporte que les autres architectures n'ont pas.

— **HITL natif** : `interrupt_before=["recruteur"]` suspend l'exécution juste avant A7, expose l'état complet à l'opérateur humain, et permet la reprise exacte après décision, sans perte de contexte.

— **Checkpointing** : chaque étape est sauvegardée. En cas de crash ou de timeout Groq, le pipeline peut reprendre à n'importe quel nœud sans tout recommencer.

— **Fan-out parallèle natif** : les primitives `Send()` et les reducers `operator.add` gèrent N instances d'A4 et d'A5 en parallèle sans code de synchronisation explicite.

— **État partagé typé** : le `TypedDict GraphState` documente formellement les contrats inter-agents et détecte les erreurs de champ statiquement.

— **Auditabilité totale** : l'historique complet des états est consultable via `app.get_state_histor` pour un post mortem.

Lancement :

Listing 5.17 – Lancement mode LangGraph (run_local.sh)

```
./run_local.sh
# Lance en parallele :
#   uvicorn src.api:app  --port 8000   (API FastAPI + LangGraph)
#   python  src/ui/app.py              (UI NiceGUI port 8080)
# LangGraph s'execute dans le meme processus que l'API FastAPI.
```

Avantages : HITL natif, checkpointing, fan-out parallèle, état typé, debugging unifié (un seul processus, tous les logs au même endroit).

Inconvénients : Un seul processus Python, si l'API tombe, tout tombe. Pas de scaling horizontal sans refonte.

5.12.2 Mode secondaire : microservices Docker

Principe. Chaque agent est déployé dans un conteneur Docker indépendant exposant une API FastAPI. Un orchestrateur HTTP (`services/orchestrateur/main.py`) remplace LangGraph : il appelle les services l'un après l'autre via HTTP POST et transmet les résultats intermédiaires au suivant. Les huit conteneurs communiquent sur un réseau Docker `bridge` interne ; les ports 8000 (API Gateway), 8080 (UI) et 8001–8006 (services individuels) sont exposés à l'hôte.

TABLE 5.6 – Services Docker (`docker-compose.microservices.yml`)

Service	Port	Rôle
svc-analyste	8001	A2 : `POST /analyser` → `profil_competences`
svc-chercheur	8002	A3a+b+c : `POST /chercher` → `profils_bruts`
svc-evaluateur	8003	A4 : `POST /evaluer` → `candidat_score`
svc-verificateur	8004	A5 : `POST /verifier` → `candidat_valide`
svc-recruteur	8005	A7 : `POST /recruter` → `messages_envoyes`
svc-orchestrateur	8006	`POST /pipeline` → coordonne tout
api-gateway	8000	Point d'entrée public (identique mode local)
sma-ui	8080	Interface NiceGUI (identique mode local)

Parallélisme asyncio dans l'orchestrateur HTTP. L'orchestrateur n'est pas purement séquentiel : il reproduit le fan-out d'A4 et d'A5 via `asyncio.gather`, appelant simultanément autant de services `svc-evaluateur` et `svc-verificateur` qu'il y a de candidats.

Listing 5.18 – Fan-out async A4 et A5 (services/orchestrateur/main.py)

```
# A4 : N evaluations en parallele via asyncio.gather
scores_raw = await asyncio.gather(*[
    _evaluer_un(client, p, profil_competences, fiche_poste)
    for p in profils
])
candidats_scores = [s for s in scores_raw if not s.get("_error")]

# A5 : N verifications en parallele
valides_raw = await asyncio.gather(*[
    _verifier_un(client, s, profil_index.get(s["candidat_id"], {}),
                 profil_competences)
    for s in candidats_scores
])
```

Le RAG (blacklist, cache, injection proactive) est actif dans les deux modes : `svc-evaluateur` appelle `get_score_cache()` avant chaque scoring Groq, et l'orchestrateur HTTP exé-

cute l'injection RAG directement avant le routage.

Lancement :

Listing 5.19 – Lancement mode microservices

```
docker-compose -f docker-compose.microservices.yml up --build
# 8 conteneurs, reseau bridge interne
# Memes ports publics qu'en mode local : 8000 (API) et 8080 (UI)
# L'API detecte SVC_ORCHESTRATEUR et bascule automatiquement
# vers l'orchestrateur HTTP au lieu de LangGraph.
```

Les deux modes sont mutuellement exclusifs à l'échelle d'une instance API : si la variable d'environnement `SVC_ORCHESTRATEUR` est définie, tous les runs utilisent l'orchestrateur HTTP ; sinon, tous utilisent LangGraph. Il n'est pas possible de mixer les deux sur la même instance.

Avantages : Chaque service est scalable et redémarrable indépendamment. La panne d'un service n'arrête pas les autres. Les services `svc-evaluateur` peuvent être dupliqués (3 instances) pour absorber un volume élevé de candidats. Déploiement sur Kubernetes directement applicable.

Limitations : Le checkpointing d'état n'est pas disponible : en cas de panne en cours de pipeline, il faut repartir de zéro. Le HITL est implémenté via `asyncio.Event` dans l'orchestrateur HTTP — le pipeline se suspend avant A7, attend une décision via `POST /hitl/{run_id}`, puis reprend — mais sans checkpointing, une panne pendant la pause HITL nécessite un redémarrage complet.

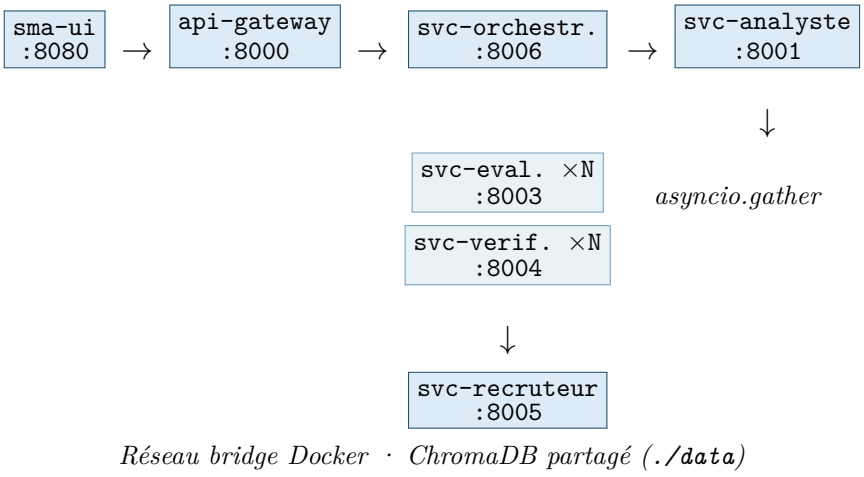


FIGURE 5.10 – Architecture microservices : 8 conteneurs Docker avec orchestration HTTP.

5.12.3 Comparaison architecturale

TABLE 5.7 – LangGraph monolithique vs microservices Docker

Critère	**LangGraph (principal)**	**Microservices Docker**
Orchestration	`StateGraph` compilé, typé	HTTP REST asyncio
État partagé	`GraphState` TypedDict	Transmission JSON inter-services
Parallélisme A4/A5	`Send()` + reducers natifs	`asyncio.gather`
HITL	Natif (`interrupt_before`)	Partiel (`asyncio.Event`, sans checkpointing)
Checkpointing	`MemorySaver` (par nœud)	**Non disponible**
RAG	Actif (blacklist + cache + injection)	Actif (identique)
Scalabilité	Mono-processus	Horizontale par conteneur
Isolation des pannes	Aucune (tout-ou-rien)	Par service
Débogage	Logs unifiés, état inspectable	Logs par conteneur
Lancement	`./run_local.sh`	`docker-compose -f docker-compose.microservices.yml up`
Usage recommandé	Développement, démo, HITL	Production, cloud, volume

Pourquoi LangGraph reste l'architecture de référence. Le mode microservices démontre la portabilité des agents : chacun vit dans son propre conteneur sans modification de sa logique métier. Le HITL y est implémenté via `asyncio.Event`, les deux modes respectent donc l'exigence EF-5. La différence reste le checkpointing : LangGraph persiste l'état complet du graphe après chaque nœud via `MemorySaver`, ce qui permet de reprendre un run interrompu sans repartir de zéro. Le mode microservices n'a pas cet équivalent ; en cas de panne, le run est perdu. LangGraph est donc le mode de référence pour la robustesse ; le mode microservices est une démonstration de scalabilité horizontale.

5.13 Tests automatisés

La suite de tests couvre 97 cas répartis sur 7 fichiers pytest. Tous les appels LLM sont mockés : les tests vérifient la logique du système (routage, filtrage, API, mémoire RAG) indépendamment de Groq, ce qui les rend déterministes et exécutables sans clé API.

TABLE 5.8 – 97 tests répartis sur 7 fichiers

Fichier	Tests	Ce qui est vérifié
`test_filtre.py`	24	Filtres URL, keywords, agrégateurs
`test_api.py`	22	Endpoints REST, HITL
`test_graph.py`	17	Topologie, 16 nœuds, routage
`test_rag.py`	16	ChromaDB, calibration, blacklist
`test_deduplicateur.py`	11	Similarité noms et URLs
`test_verificateur_rules.py`	5	Guards déterministes A5
`test_stratege_queries.py`	2	Structure requêtes A3a
Total	**97**	**97/97 passent**

Logique de routage (17 tests). `test_graph.py` vérifie que le graphe LangGraph contient bien les 16 nœuds attendus, que `interrupt_before=["recruteur"]` est déclaré, et surtout que le routage conditionnel après A5 est correct : un score ≥ 75 oriente vers A7, un score < 75 mais avec 3 candidats viables ≥ 40 aussi (mode relatif), et l'absence de candidats qualifiés saute directement au rapport. Ces cas limites (exactement 75, 3 viables, tous invalides) sont chacun un test séparé.

Listing 5.20 – test_graph.py — cas limites du routage conditionnel

```python
def test_score_exactement_75_route_recruteur(self):
    state = {"candidats_valides": [_valide(score=75.0)]}
    assert route_apres_verification(state) == "recruteur"

def test_douteux_score_eleve_ne_route_pas_recruteur(self):
    """Un profil douteux ne doit jamais etre contacte
       automatiquement."""
    state = {"candidats_valides": [_valide(score=90.0, statut="
       douteux")]}
    assert route_apres_verification(state) == "rapport"

def test_aucun_candidat_route_rapport(self):
    state = {"candidats_valides": []}
    assert route_apres_verification(state) == "rapport"
```

API et HITL (22 tests). `test_api.py` couvre les six endpoints REST. Les cas d'erreur sont explicitement testés : fiche vide retourne 422, run inconnu retourne 404, décision HITL sur un run qui n'est pas en attente retourne 409, action HITL invalide retourne 422. Les cas nominaux vérifient que `approve` libère bien le worker

(`threading.Event.set()`), que `skip` enregistre la décision, et que `edit` persiste la liste de candidats modifiée.

Listing 5.21 – test_api.py — HITL : erreurs et cas nominal approve

```python
def test_hitl_run_pas_en_attente_409(self, client):
    _runs[run_id] = _minimal_run(run_id, status="running")
    resp = client.post(f"/runs/{run_id}/hitl", json={"action": "
        approve"})
    assert resp.status_code == 409

def test_hitl_approve_signale_le_worker(self, client):
    _runs[run_id] = _minimal_run(run_id, status="awaiting_hitl")
    assert not _runs[run_id]["_hitl_event"].is_set()

    resp = client.post(f"/runs/{run_id}/hitl", json={"action": "
        approve"})

    assert resp.status_code == 200
    assert _runs[run_id]["_hitl_event"].is_set()       # worker
        debloque
    assert _runs[run_id]["_hitl_decision"]["action"] == "approve"
```

Mémoire RAG (16 tests). `test_rag.py` teste ChromaDB en base temporaire isolée (répertoire `tmp` créé par fixture). On vérifie l'upsert idempotent sur un même `candidat_id`, la recherche par similarité cosinus, l'isolation par fiche de poste (un candidat persisté pour la fiche A n'apparaît pas dans les résultats de la fiche B), et la blacklist inter-runs.

Listing 5.22 – test_rag.py — upsert idempotent et isolation par fiche

```python
def test_upsert_meme_id(self, memoire):
    """Ajouter deux fois le meme ID ne cree pas de doublon."""
    memoire.ajouter_candidat("c1", "Alice", "Python dev", 70.0, "
        github")
    memoire.ajouter_candidat("c1", "Alice", "Python dev senior",
        82.0, "github")
    assert memoire.compter() == 1    # toujours 1 seul
        enregistrement

def test_rag_isole_par_fiche(self, memoire):
    """Fiche courante proche de fiche_A : ne remonte pas les
        candidats fiche_B."""
    fid_py = memoire.ajouter_fiche_poste("Dev Python ML Paris
        senior")
    fid_mk = memoire.ajouter_fiche_poste("Chef projet marketing
        digital B2B Lyon")
```

```
    memoire.ajouter_candidat("c_py", "Alice", "Python ML", 85.0, "github",
                             fiche_id=fid_py)
    memoire.ajouter_candidat("c_mk", "Bob",   "Marketing B2B", 80.0, "linkedin",
                             fiche_id=fid_mk)
    # Recherche pour une fiche Python : Bob (marketing) ne doit pas remonter
    res = memoire.rechercher_similaires("Python ML", fiche_poste="Dev Python Paris")
    noms = [r["nom"] for r in res]
    assert "Bob" not in noms
```

Filtrage et déduplication (35 tests). `test_filtre.py` couvre les règles déterministes du filtre A3c : URL de bruit (pages entreprises, agrégateurs, annonces), texte hors-sujet, profils trop généralistes. `test_deduplicateur.py` vérifie la similarité entre noms (translittération, accents) et entre URLs (query strings différents, même profil).

Listing 5.23 – test_filtre.py — distinction offre d'emploi / profil candidat

```
def test_offre_emploi_detectee(self):
    text = "Nous recherchons un developpeur Python pour rejoindre notre equipe"
    assert _is_noise_text(text) is True    # page d'offre -> bruit

def test_profil_candidat_autorise(self):
    text = "Alice Dupont, developpeuse Python 7 ans, specialiste ML"
    assert _is_noise_text(text) is False  # profil -> conserve

def test_search_page_bloquee(self):
    assert _is_noise_url("https://www.simplyhired.fr/search?q=dev")
        is True

def test_linkedin_autorise(self):
    assert _is_noise_url("https://www.linkedin.com/in/alice-dupont"
    ) is False
```

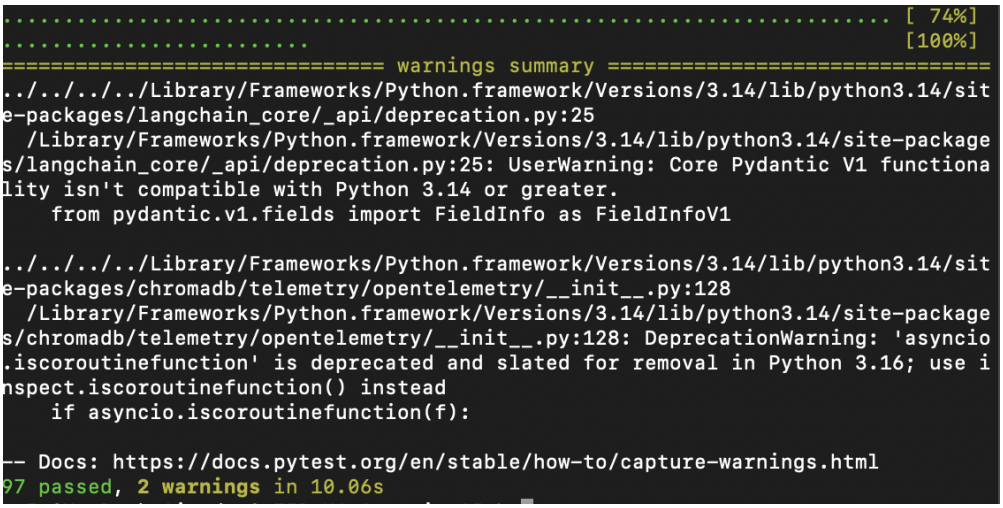


FIGURE 5.11 – Résultat `pytest` : 97/97 tests passent.

Chapitre 6

Évaluation et résultats

6.1 Run de référence (Run 1) — base ChromaDB vide

Pour valider concrètement le système, on a lancé un premier run complet sur une fiche de poste réelle : *"Développeur Python senior, 5 ans d'expérience, Paris ou remote, FastAPI, LangGraph, Docker, RAG, bonne communication"*. Le modèle utilisé est Groq `llama-3.3-70b-versatile`. La base ChromaDB est vide : c'est le run d'amorçage, sans mémoire inter-runs disponible.

Table 6.1 – Métriques du Run 1 — base ChromaDB vide

Étape	**Valeur**	**Commentaire**
Profils bruts collectés (A3b)	46	DDG + GitHub + SO
Après filtrage A3c	13	Taux de bruit : 68%
Après déduplication A6	12	1 doublon nom
Candidats évalués A4	10	Limite MAX_PROFILS=10
Candidats valides A5	5	Dont 3 upgrades LinkedIn
Candidats injectés RAG	0	Base vide
Messages rédigés A7	2	Score ≥ 75
Durée totale	**173 s**	10 appels LLM parallèles

Table 6.2 – Meilleurs candidats identifiés au Run 1

Candidat	Score	Source	Remarque A5
Srikanth Vanjre	88	LinkedIn	Senior Backend AI/RAG
Shubhangi Gaur	80	LinkedIn	Python/FastAPI senior
Nikita Koznev	72	LinkedIn	Python senior (upgrade auto)
Jyotsna Konchada	68	LinkedIn	Bon profil Python/ML
Vamshi Krishna	68	LinkedIn	Bon profil Python

Ces cinq candidats sont persistés dans ChromaDB par A8 à la fin du run. Ils constituent la mémoire de référence pour les runs suivants.

6.2 Run 2 — premier apport du RAG (base peuplée)

Le deuxième run est exécuté immédiatement après le run de référence, sur la même fiche de poste. La base ChromaDB contient désormais les cinq candidats persistés au Run 1 : c'est le premier run où les trois mécanismes RAG s'activent réellement.

Table 6.3 – Métriques du Run 2 — base ChromaDB peuplée (Run ID : 20260523_022804)

Étape	Valeur	Commentaire
Profils bruts collectés (A3b)	53	DDG + GitHub + SO
Après dédup interne A3b	44	Dédup URL avant filtre
Après filtrage A3c	15	Taux de bruit : 72%
Après déduplication A6	14	1 doublon nom
Candidats évalués A4	10	Aucun cache hit
Candidats injectés RAG	5	En 0,05 s, sans LLM
Candidats valides A5 (live)	1	Mickael Douabin (80)
Candidats valides total	**6**	1 live + 5 RAG
Messages rédigés A7	3	Mode absolu (score ≥ 75)
Durée totale	**155,5 s**	-10% vs Run 1

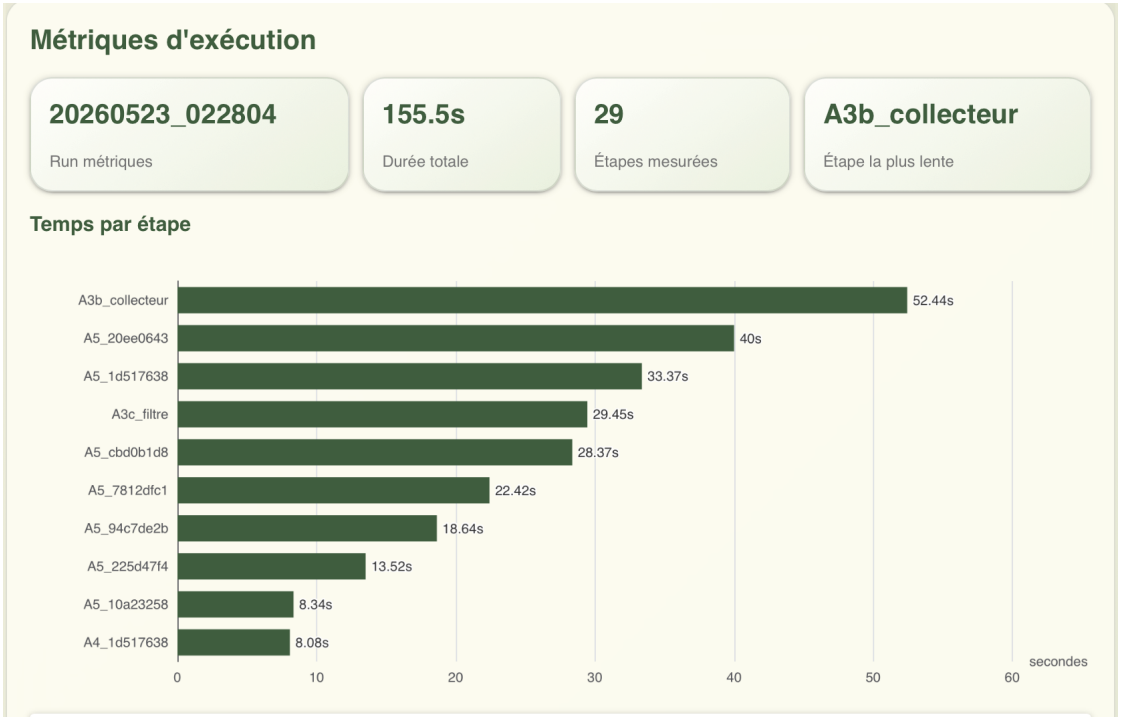
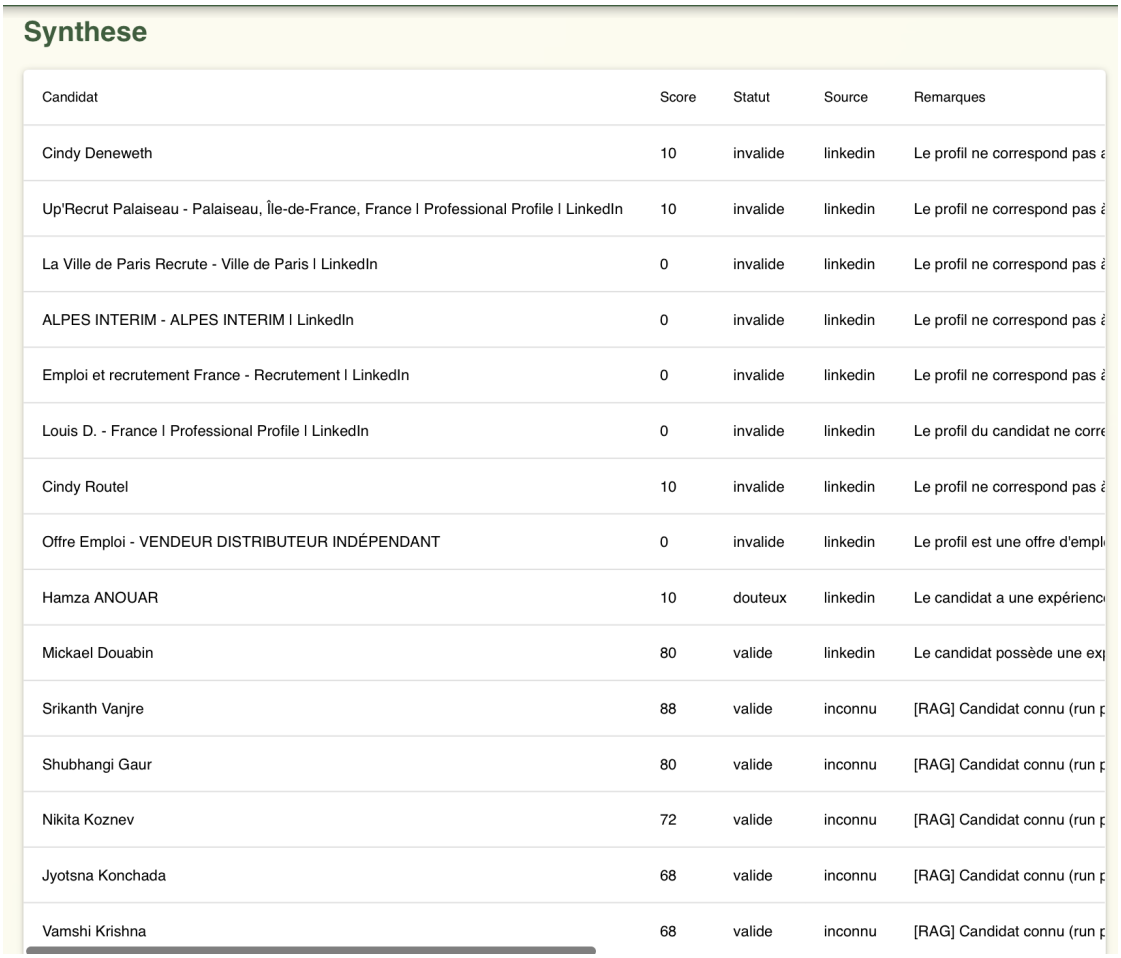


FIGURE 6.1 – Page Métriques du Run 2 : tableau d'exécution avec durée totale 155,5 s et mesures par étape (A3b Collecteur, A4, injection_rag).

TABLE 6.4 – Candidats identifiés au Run 2 (valides et douteux)

Candidat	**Score**	**Source**	**Remarque A5**
Srikanth Vanjre	88	RAG	Senior Backend AI/RAG — profil validé run précédent
Shubhangi Gaur	80	RAG	Python/FastAPI senior — profil validé run précédent
Mickael Douabin	80	LinkedIn	Ingénieur senior Python/FastAPI — validé live
Nikita Koznev	72	RAG	Python senior — profil validé run précédent
Jyotsna Konchada	68	RAG	Bon profil Python/ML — profil validé run précédent
Vamshi Krishna	68	RAG	Bon profil Python — profil validé run précédent
Hamza ANOUAR	10	LinkedIn	Douteux : compétences non prouvées



Synthese

Candidat	Score	Statut	Source	Remarques
Cindy Deneweth	10	invalide	linkedin	Le profil ne correspond pas a
Up'Recrut Palaiseau - Palaiseau, Île-de-France, France	Professional Profile	LinkedIn	10	invalide	linkedin	Le profil ne correspond pas à
La Ville de Paris Recrute - Ville de Paris	LinkedIn	0	invalide	linkedin	Le profil ne correspond pas à
ALPES INTERIM - ALPES INTERIM	LinkedIn	0	invalide	linkedin	Le profil ne correspond pas à
Emploi et recrutement France - Recrutement	LinkedIn	0	invalide	linkedin	Le profil ne correspond pas à
Louis D. - France	Professional Profile	LinkedIn	0	invalide	linkedin	Le profil du candidat ne corre
Cindy Routel	10	invalide	linkedin	Le profil ne correspond pas à
Offre Emploi - VENDEUR DISTRIBUTEUR INDÉPENDANT	0	invalide	linkedin	Le profil est une offre d'empl
Hamza ANOUAR	10	douteux	linkedin	Le candidat a une expérienc
Mickael Douabin	80	valide	linkedin	Le candidat possède une ex
Srikanth Vanjre	88	valide	inconnu	[RAG] Candidat connu (run p
Shubhangi Gaur	80	valide	inconnu	[RAG] Candidat connu (run p
Nikita Koznev	72	valide	inconnu	[RAG] Candidat connu (run p
Jyotsna Konchada	68	valide	inconnu	[RAG] Candidat connu (run p
Vamshi Krishna	68	valide	inconnu	[RAG] Candidat connu (run p

FIGURE 6.2 – Page Rapport du Run 2 : 6 candidats valides dont 5 réinjectés depuis la mémoire RAG.

Observations du Run 2. Injection RAG en 0,05 s. Les 5 candidats validés au Run 1 sont réinjectés sans aucun appel LLM. La similarité cosine entre les deux fiches identiques dépasse le seuil de 0,75, ce qui déclenche l'injection proactive.

Nouveau candidat live. Mickael Douabin (Ingénieur senior Python/FastAPI) n'était pas dans la base. A4 lui attribue 85/100 en 8,08 s ; A5 le confirme à 80 avec statut `valide`.

Routage absolu. Le meilleur score (88,0) dépasse `SCORE_SEUIL_CONTACT` = 75. A7 rédige 3 messages de contact en 1,18 s, contre 2 au Run 1.

6.3 Run 3 — accumulation de la mémoire

Le troisième run est exécuté après le Run 2, sur la même fiche. La base ChromaDB est désormais enrichie de six candidats : les cinq du Run 1 et Mickael Douabin, validé live au Run 2. C'est ce run qui permet de mesurer l'accumulation de la mémoire sur

plusieurs exécutions successives.

TABLE 6.5 – Métriques du Run 3 — base ChromaDB enrichie (Run ID : 20260523_024521)

Étape	**Valeur**	**Commentaire**
Profils bruts collectés (A3b)	32	DDG + GitHub + SO
Après filtrage A3c	15	Taux de bruit : 53%
Après déduplication A6	15	Aucun doublon
Candidats évalués A4	10	
Scores A4 réutilisés (cache)	1	Mickael Douabin : 0,35 s vs 8,08 s
Candidats injectés RAG	5	En 0,05 s, sans LLM
Candidats valides total	5	Tous issus du RAG
Messages rédigés A7	2	Mode absolu (score $\geq$ 75)
Durée totale	**138,1 s**	−20% vs Run 1

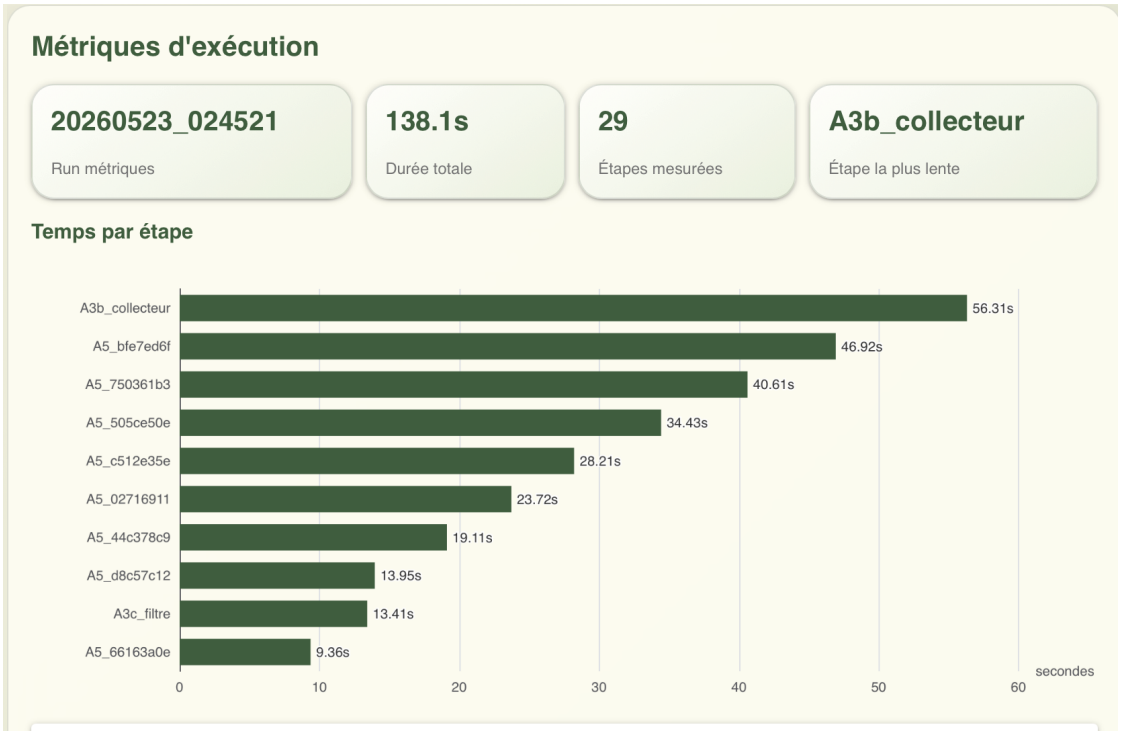
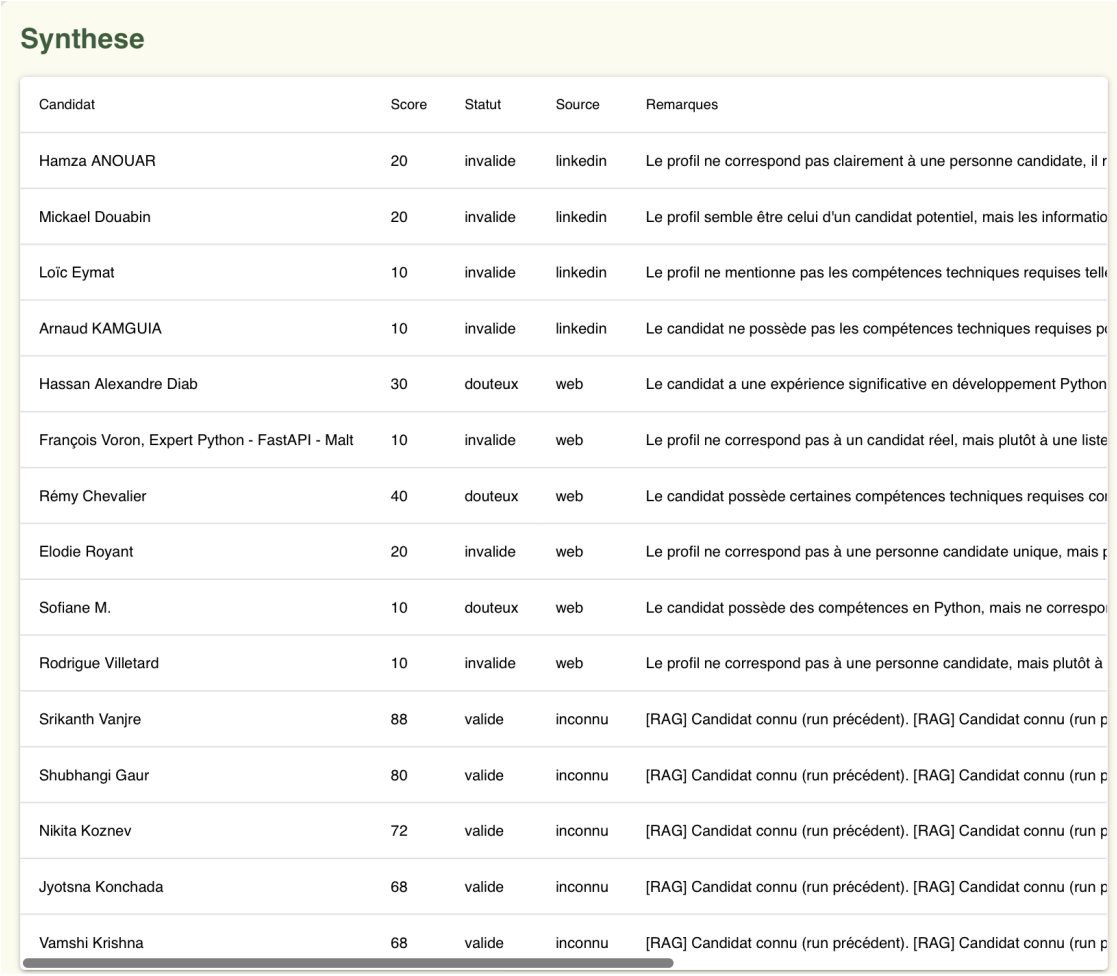


FIGURE 6.3 – Page Métriques du Run 3 : tableau d'exécution avec durée totale 138,1 s — on observe le cache hit A4 (0,35 s) et l'injection RAG (0,05 s).



Synthese

Candidat	Score	Statut	Source	Remarques
Hamza ANOUAR	20	invalide	linkedin	Le profil ne correspond pas clairement à une personne candidate, il r
Mickael Douabin	20	invalide	linkedin	Le profil semble être celui d'un candidat potentiel, mais les informatio
Loïc Eymat	10	invalide	linkedin	Le profil ne mentionne pas les compétences techniques requises telle
Arnaud KAMGUIA	10	invalide	linkedin	Le candidat ne possède pas les compétences techniques requises pe
Hassan Alexandre Diab	30	douteux	web	Le candidat a une expérience significative en développement Python
François Voron, Expert Python - FastAPI - Malt	10	invalide	web	Le profil ne correspond pas à un candidat réel, mais plutôt à une liste
Rémy Chevalier	40	douteux	web	Le candidat possède certaines compétences techniques requises cor
Elodie Royant	20	invalide	web	Le profil ne correspond pas à une personne candidate unique, mais p
Sofiane M.	10	douteux	web	Le candidat possède des compétences en Python, mais ne correspo
Rodrigue Villetard	10	invalide	web	Le profil ne correspond pas à une personne candidate, mais plutôt à
Srikanth Vanjre	88	valide	inconnu	[RAG] Candidat connu (run précédent). [RAG] Candidat connu (run p
Shubhangi Gaur	80	valide	inconnu	[RAG] Candidat connu (run précédent). [RAG] Candidat connu (run p
Nikita Koznev	72	valide	inconnu	[RAG] Candidat connu (run précédent). [RAG] Candidat connu (run p
Jyotsna Konchada	68	valide	inconnu	[RAG] Candidat connu (run précédent). [RAG] Candidat connu (run p
Vamshi Krishna	68	valide	inconnu	[RAG] Candidat connu (run précédent). [RAG] Candidat connu (run p

FIGURE 6.4 – Page Rapport du Run 3 : les 5 candidats valides sont tous issus de la mémoire RAG.

Observations du Run 3. Cache A4 : 0,35 s au lieu de 8,08 s. Mickael Douabin étant déjà évalué au Run 2, A4 retrouve son score (80) depuis le cache RAG en 0,35 s. Cependant, A5 réévalue le profil brut indépendamment et le classe cette fois `invalide` (20/100) : le contenu scrapé est plus limité qu'au Run 2 (LinkedIn a renvoyé un aperçu incomplet). C'est le comportement attendu du pattern validation pair-à-pair : A5 ne fait jamais confiance au cache d'A4, il contrôle toujours.

Réduction de durée progressive. 173 s → 155,5 s → 138,1 s. L'essentiel du gain vient de l'injection RAG (5 candidats en 0,05 s) et, au Run 3, du cache A4 qui court-circuite un appel Groq.

6.4 Synthèse comparative des trois runs

TABLE 6.6 – Évolution des métriques sur trois runs successifs — même fiche de poste

Métrique	**Run 1**	**Run 2**	**Run 3**
Profils bruts A3b	46	53	32
Après filtrage A3c	13	15	15
Scores A4 réutilisés (cache)	0	0	1
Candidats injectés RAG	0	5	5
Candidats valides total	5	6	5
Messages rédigés A7	2	3	2
Durée totale	**173 s**	**155,5 s**	**138,1 s**
Gain vs Run 1	—	−10%	−20%

Le RAG calibré par fiche de poste transforme chaque exécution en apprentissage pour les suivantes. Dès le Run 2, 5 candidats sont réinjectés sans aucun appel LLM et un nouveau candidat live est détecté. Au Run 3, le cache A4 s'active en plus. Ces gains s'accumulent : chaque run enrichit la base et réduit le coût du suivant, c'est la principale valeur ajoutée par rapport à un pipeline stateless.

Chapitre 7

Conclusion et perspectives

7.1 Bilan du projet

Ce TER a duré plusieurs mois, du cadrage théorique jusqu'à la mise en production du prototype. En partant d'une question simple — peut-on automatiser la première phase du recrutement sans perdre la maîtrise humaine des décisions ? — nous avons construit un système qui répond concrètement à cette question.

Ce qu'on a fait. La première partie du travail a été théorique : lire et synthétiser 17 études sur les Systèmes Multi-Agents et l'Agentic AI, comprendre ce que chaque approche apporte et où elle échoue. Ce travail n'était pas un exercice académique abstrait — il a directement orienté nos choix d'architecture. C'est en lisant ces études qu'on a compris pourquoi les LLM seuls sont insuffisants pour des décisions binaires, pourquoi le HITL doit être garanti architecturalement et pas seulement documenté, et pourquoi la séparation des agents par responsabilité rend le système testable et maintenable.

La deuxième partie a été la comparaison de neuf frameworks d'orchestration. Nous avons construit une grille de onze critères ancrés dans les besoins réels du projet (graphe orienté, état typé, HITL natif, RAG, indépendance LLM) et attribué des scores à chaque framework. LangGraph a obtenu 51/55 — non pas par défaut, mais parce qu'il est le seul à cocher toutes les cases critiques pour un pipeline de recrutement responsable.

La troisième partie a été l'implémentation, la plus longue et la plus instructive. Le résultat est un prototype complet : 10 agents spécialisés, une mémoire vectorielle inter-runs (ChromaDB), un point de contrôle humain obligatoire, une API REST avec streaming temps réel, une interface web six pages (NiceGUI), deux architectures d'exécution

(LangGraph monolithique et microservices Docker), et 97 tests automatisés.

Ce que le prototype démontre. Le système répond aux sept exigences fonctionnelles du cahier des charges : sourcing sans API payante (EF-1), filtrage algorithmique du bruit (EF-2), scoring avec explications (EF-3), validation pair-à-pair A4/A5 (EF-4), HITL obligatoire avant tout contact (EF-5), mémoire inter-runs calibrée par fiche (EF-6), et suivi en temps réel (EF-7).

Mapping théorie → implémentation

Le tableau ci-dessous montre concrètement comment chaque concept étudié dans l'état de l'art se retrouve dans une décision d'implémentation précise.

TABLE 7.1 – Correspondance entre concepts SMA/Agentic AI et implémentation

Concept théorique	**Paradigme**	**Implémentation**
Blackboard partagé	SMA classique	`GraphState` TypedDict avec droits d'écriture séparés
Pattern Superviseur	SMA classique	A1 coordonne le pipeline et génère le rapport final
Validation pair-à-pair	SMA classique	A4 évalue / A5 vérifie indépendamment
Contract Net Protocol	SMA classique	Fan-out `Send()` → N instances A4 et A5
Réducteurs de consensus	SMA classique	`Annotated[list, operator.add]` sur les champs parallèles
Modèle BDI	SMA classique	GraphState (beliefs), missions agents (desires), appels LLM (intentions)
Architecture modulaire	Agentic AI	10 agents spécialisés (A1–A8, A3 décomposé en A3a/b/c) + nœuds techniques
Mémoire long terme (RAG)	Agentic AI	ChromaDB 2 collections calibrées par fiche de poste
Tool calling	Agentic AI	DDG, GitHub API, Stack Overflow API, scraping HTTP
Human-in-the-Loop	Agentic AI	`interrupt_before=["recruteur"]` + `threading.Event`
Paradigme neuro-symbolique	Hybride	LLM pour la cognition, guards déterministes pour les décisions critiques
Niveau 3 d'agenticité	Agentic AI	Orchestration autonome multi-agents avec HITL intégré

7.2 Ce qu'on a appris

Ce projet nous a appris plusieurs choses qu'on ne pouvait pas deviner en lisant les articles.

La première leçon est que **les LLM seuls ne suffisent pas**. Les premières versions du prototype, où A5 faisait confiance au jugement du modèle sans garde-fous, produisaient des résultats instables : des pages d'entreprises notées comme candidats, des profils LinkedIn bloqués acceptés sur la base d'un titre trompeur, des scores qui variaient de 20 points d'un run à l'autre pour le même profil. L'ajout progressif des cinq guards déterministes d'A5 a été le changement le plus impactant du projet. Ce n'est pas une critique des LLM — c'est une confirmation du paradigme neuro-symbolique : les LLM excellent là où la tâche est floue et cognitive, les règles déterministes excellent là où la décision doit être binaire et auditables.

La deuxième leçon est que **la séparation des responsabilités est indispensable** dès qu'on veut tester. L'agent A3 était initialement un seul agent qui faisait stratégie, collecte et filtrage. C'était difficile à déboguer (impossible de savoir si un problème venait de la génération des requêtes ou du scraping) et impossible à tester unitairement. Le refactoring en A3a/A3b/A3c a été coûteux en temps mais il a rendu le code testable et chaque couche modifiable indépendamment.

La troisième leçon est que **le HITL doit être garanti par l'architecture, pas par la documentation**. Utiliser `interrupt_before` de LangGraph garantit qu'aucun contact candidat ne peut avoir lieu sans décision humaine explicite, même si un développeur oublie de vérifier. C'est la différence entre une contrainte architecturale et une simple convention.

La quatrième leçon concerne le **RAG**. On pensait au départ que la mémoire inter-runs était une fonctionnalité secondaire. En pratique, c'est ce qui transforme le système d'un pipeline stateless en un système qui apprend : au troisième run, cinq candidats déjà validés sont réinjectés sans appel LLM grâce au RAG (0,05 s contre plusieurs secondes par évaluation), un score A4 est réutilisé depuis le cache (0,35 s contre 8,08 s), et la durée totale baisse d'environ 20% par rapport au run de référence. Cette valeur s'accumule à chaque run.

7.3 Limites

Hallucinations et non-déterminisme des LLM

Les agents A4 et A5 reposent sur Groq (`llama-3.3-70b-versatile`), un LLM probabiliste. Deux appels identiques peuvent produire des scores différents, et

le modèle peut halluciner, attribuer un score élevé à un profil hors sujet ou inversement rejeter un bon candidat. Les guards déterministes d'A5 atténuent ce risque mais ne l'éliminent pas.

Fragilité du scraping

DuckDuckGo peut retourner des résultats vides ou bloquer les requêtes trop fréquentes. LinkedIn bloque quasi-systématiquement le scraping direct : les profils `/in/` ne fournissent que le snippet DDG, ce qui prive A4 et A5 du texte complet. L'upgrade automatique d'A5 compense partiellement, mais le bassin de candidats reste biaisé vers les profils techniques publics (GitHub, Stack Overflow). En production, une API LinkedIn officielle serait nécessaire.

Quota Groq et dépendance LLM externe

Le *free tier* Groq est limité à 100 000 tokens/jour, soit 3 à 5 runs complets. Au-delà, le pipeline retourne des erreurs 429. Le cache RAG réduit la consommation sur les runs répétés, mais la contrainte reste présente pour les premières utilisations.

7.4 Considérations éthiques et RGPD

Le recrutement automatisé est classé parmi les systèmes à haut risque par l'AI Act européen. Trois points méritent d'être soulevés. D'abord, les LLM héritent des biais de leurs données d'entraînement — les guards déterministes d'A5 réduisent ce risque sans l'éliminer. Ensuite, les profils collectés sont des données personnelles au sens du RGPD : le stockage est local (ChromaDB), aucun contact n'est initié sans validation humaine (HITL), mais les profils transitent par l'API Groq lors de l'évaluation. En production, passer à un modèle local via `LLM_PROVIDER=ollama` supprimerait cette transmission. Enfin, le système est conçu comme un outil d'aide à la décision : la décision finale reste humaine, ce que le HITL obligatoire garantit architecturalement.

Bibliographie

[1] Maldonado, J. et al. (2024). *Architectures and Coordination Patterns in Multi-Agent Systems*. JAIR, 79.

[2] Boissier, O. (s.d.). *Cours Systèmes Multi-Agents*. École des Mines de Saint-Étienne.

[3] Zhang, W. et al. (2020). *A General Survey on Multi-Agent Coordination Mechanisms*. ACM Computing Surveys, 53(4).

[4] Gronauer, S., Diepold, K. (2021). *Multi-Agent Deep Reinforcement Learning : A Survey*. Artificial Intelligence Review, 55.

[5] Nguyen, T. et al. (2019). *Deep Reinforcement Learning for Multi-Agent Systems*. IEEE Transactions on Cybernetics, 50(9).

[6] (2024). *Multi-Agent Systems : The Future of Distributed AI*. World Journal of Advanced Research and Reviews (WJARR).

[7] Schneider, J. (2025). *Generative to Agentic AI : Survey, Conceptualization, and Challenges*. arXiv.

[8] Sapkota, B. et al. (2026). *AI Agents vs. Agentic AI : A Conceptual Taxonomy*. arXiv.

[9] Hughes, A. et al. (2025). *Safety and Alignment Challenges in Autonomous AI Agents*. Nature Machine Intelligence.

[10] Wavestone (2026). *Agentic AI Playbook*.

[11] Abou Ali, H. et al. (2026). *Symbolic vs Neural Paradigms in Agentic AI*. Expert Systems with Applications.

[12] Milosevic, D., Rabhi, F. (2026). *Architecting Agentic Communities using Design Patterns*. IEEE Software.

[13] Vaidhyanathan, K., Taibi, D. (2026). *Agentic AI Frameworks Under the Microscope*. IEEE Software, 43(1), 133–138.

[14] Microsoft Research (2024). *AutoGen Documentation*. https://microsoft.github.io/autogen/

[15] CrewAI Inc. (2024). *CrewAI Documentation*. https://docs.crewai.com/

[16] LangChain Inc. (2024). *LangGraph Documentation*. https://langchain-ai.github.io/langgraph/

[17] Chroma (2024). *ChromaDB Documentation*. https://docs.trychroma.com/

[18] Ramírez, S. (2024). *FastAPI Documentation*. https://fastapi.tiangolo.com/

[19] zauberzeug (2024). *NiceGUI Documentation*. https://nicegui.io/